Re-Tokenization, Segmentation, and a Blind Spot: What Content-Free Boundary Markers Do in Character-Level Transformations

Anonymous authors
Paper under double-blind review

Abstract

Inserting a content-free marker (|) into the input every k characters of a character-level transformation, and asking the model to copy it through, is a cheap single-call intervention. We study what it does across seven tasks, seven open instruction-tuned models, and Gemini 3.5 Flash, using two controls that fix tokenization at one token per character: the same letter tasks with a space between letters, and exact relabelings with digits. We find three separate effects. First, on letter strings, which tokenizers merge into multi-letter tokens, markers act largely as re-tokenization: at five of the seven open models, spacing the letters with no markers at all improves accuracy more (+0.16 to +0.33) than markers do (+0.13 to +0.26), and the large advantage of the position right after a marker disappears once letters are spaced. Second, the gain survives when each character is already its own token: with spaced letters or digits they remove 33% to 83% of the remaining errors at the four strongest open models, with little dependence on spacing; at Qwen2.5-32B the gain persists to $N = 256$ and is the same with commas or line breaks as markers. Separating the two halves of the intervention, with the output format enforced, shows that neither side helps alone and both hurt: only markers on both sides help, while the cross-boundary failure follows the input markers by themselves. Blocking marker-attending heads removes part of it and matched controls remove none, so marker-directed attention contributes, but the mechanism is unresolved. Third, with a marker every two or more characters, the position right after a marker fails whenever its output depends on the input before the marker: on four dependency tasks it falls below the no-marker run in 29 of 31 task–model cells, to near chance at the stronger models, while the next position recovers. Commas and line breaks produce the same loss, and telling the model that the rule continues across the marker narrows it without removing it. In practice: separate characters before adding markers, use sparse markers on long sequences, and avoid placing a marker where an output needs the input just before it.

1 Introduction

Ask a language model to apply a simple, fully specified rule to every character of a string (shift each letter forward, substitute through a lookup table, output the XOR of each digit with the one before) and accuracy does not fade gracefully with length. It holds near ceiling on short inputs and then collapses well before any obvious capacity limit (Hou et al., 2026). Standard remedies change the computation itself: splitting the input across several calls (Zhou et al., 2023), asking for intermediate steps (Nye et al., 2021; Chen et al., 2023), or adding memory to the architecture (Dai et al., 2019; Bulatov et al., 2022; Mohtashami & Jaggi, 2023). Each costs extra calls, extra tokens, or training.

We study a cheaper intervention. A content-free marker character (|) is inserted into the input every k characters, and the model is told to copy it through to the output unchanged. The marker carries no task information, needs no extra call, and requires no training. On letter tasks it raises accuracy substantially, by up to +0.26 on average at the stronger models we test. Our question is why, and when it hurts.

A marker does more than mark a boundary. Tokenizers merge runs of letters into multi-letter tokens, and a marker after every few letters splits those runs, so on letter strings a marker is also a tokenization intervention. We separate the two with two controls that keep one token per character whether or not markers are present: the same letter tasks with a space between letters, and exact relabelings of them with digits. We find three effects (Figure 1):

1. **On letter strings, markers largely re-tokenize** (Section 4). At all five Qwen and Gemma models, spacing the letters with no markers at all helps more than markers on unspaced letters (+0.16 to +0.33 against +0.13 to +0.26); at Mistral and Phi the two help about equally. The striking accuracy of the position right after a marker on letter tasks (positive in all 24 task–model cells, with per-model means of +0.07 to +0.34 over the no-marker run at the same positions) is largely a token-boundary effect: with spaced letters or digits that position is no better than the rest of its segment.

2. **The gain persists when each character is its own token** (Section 5). With spaced letters or digits, markers remove 33% to 83% of the remaining errors at Qwen2.5-14B and 32B, Qwen3.8-27B, and Gemma-4-31B (significant in 23 of 24 cells), and the two controls agree closely. The gain is absent at short lengths, where these models are near ceiling, appears at $N = 32$, and at $N = 64$ reaches +0.27 (spaced letters) and +0.22 (digits), with little dependence on spacing. At Qwen2.5-32B it persists to $N = 256$ (56% of remaining errors removed) and is the same with commas or line breaks as markers. At the three smallest models the gains are smaller and inconsistent (significant in 7 of 18 cells).

3. **The position after a marker fails on dependencies that cross it** (Section 6). With a marker every two or more characters, on tasks where each output also depends on the previous input, the position right after a marker falls below the no-marker run in 29 of 31 task–model cells (every cell at the five strongest models), to near chance at the stronger models, even where the model is otherwise near ceiling. The next position recovers immediately, and rewriting the tasks so that this position needs nothing before the marker repairs it. The loss is not created by how the instruction describes the markers: telling the model they are only visual separators and that the rule continues across them narrows it but does not remove it, and commas or line breaks in place of | leave the position at chance up to $N = 256$. Pooled task-level effects on these tasks are consistent with a mix of this loss and the within-segment gains of Finding 2, which is why their sign varies with length and model.

We also test the obvious mechanism (Section 7). In Qwen2.5, some heads attend from each position to the adjacent marker, but at Qwen2.5-32B and $N = 64$, blocking the ten most marker-attending heads removes 1% of the gain on digits, 13% on spaced letters, and 19% on contiguous letters, against at most 5% for controls. Blocking 200 heads removes 21% (digits) and 28% (spaced letters), against none for controls of the same size. Marker-directed attention therefore contributes to the gain, and it is not concentrated in the few heads that attend to markers most; but the share removed does not rise monotonically with the number of heads blocked (8%, 1%, 28% on spaced letters), so these experiments do not establish how the contribution is distributed. A 2×2 separating input from output markers, with the format enforced by constrained decoding (Section 7.1), finds that marking either side alone *hurts* (−0.03 to −0.16) while marking both helps (+0.10 and +0.39), an interaction of +0.19 and +0.70; the blind spot, by contrast, follows the input markers alone. Section 8 turns the findings into guidance: separate characters before adding markers, use sparse markers on long sequences, and avoid placing a marker where an output needs the input just before it.

Supporting analyses. Each finding is backed by checks reported in the appendix: re-scoring with an edit-distance alignment, which forgives dropped or inserted characters, leaves the blind spot intact (Table 11); sampling instead of greedy decoding changes neither the gain nor the blind spot (Table 21); the per-position comparisons hold model by model and by distance from the marker (Tables 9 and 10); on third-party tasks markers help a position-local substitution task and hurt a prefix-dependent algorithm (Table 13); changing the marker symbol, jittering its placement, or rephrasing the instruction changes the size of the letter-task gain at Qwen2.5-32B but not its sign, except for a reserved special token and a terse cipher-style prompt

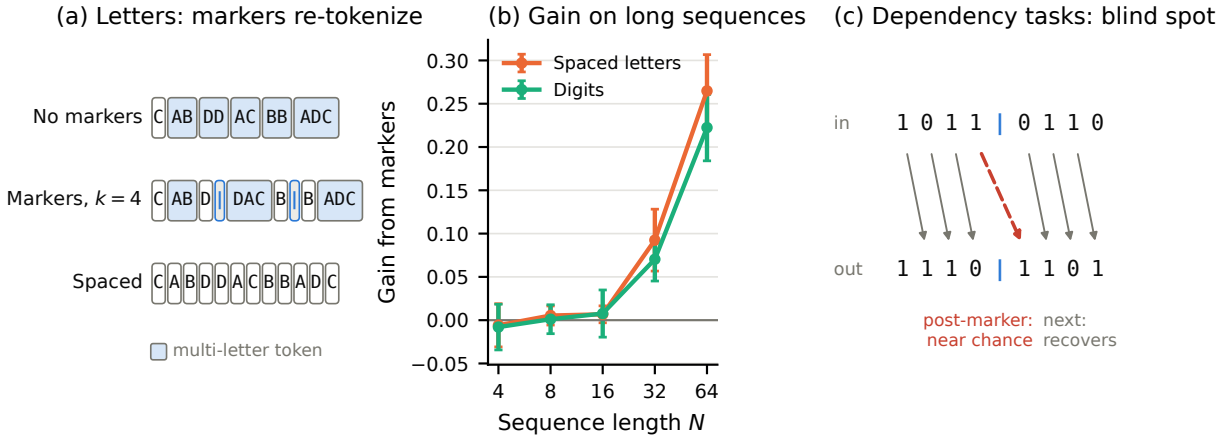


Figure 1: The three effects at a glance. (a) Qwen2.5's tokenizer merges runs of letters into multi-letter tokens; a marker every four letters splits only some of them, while spacing the letters gives one token per letter. (b) With one token per character (spaced letters or digits), markers still help the four strongest open models, from $N = 32$ on (the marker runs, "Bounds", minus the no-marker runs, "Base"; mean over models and the three position-local tasks; whiskers are 95% intervals; same data as Figure 4a). (c) On `running_xor`, each output is the XOR of its input and the one before; the position right after a marker, whose dependency crosses the marker, falls to near chance at the stronger models, and the next position recovers.

(Appendix I); and sparse markers take less wall-clock time than none, because they keep models from generating past the required length (Table 27).

2 Related work

Tokenization, formatting, and position. Subword tokenizers obscure the characters inside a token: models know only partially which characters their tokens contain (Kaushal & Mahowald, 2022), character-level manipulation is a known weakness (Edman et al., 2024), and on a large benchmark of character-level tasks, longer tokens obscure where a character sits (Uzan & Pinter, 2026). The same string can be encoded many ways, and models handle non-canonical encodings surprisingly well; splitting text into characters can even improve string manipulation (Zheng et al., 2025), and retokenizing a prompt while preserving its bytes is a clean probe of segmentation sensitivity (Jain et al., 2026). Surface formatting matters more broadly: choices of separators, spacing, and casing in a prompt can change few-shot accuracy by tens of points (Sclar et al., 2024). How many tokens a text costs also varies sharply across languages and formats, with consequences for accuracy and price (Petrov et al., 2023; Ahia et al., 2023). Formatting and positional cues change arithmetic accuracy: Nogueira et al. (2021) show that the surface form of numbers matters, Shen et al. (2023) that positional description does, and Duan et al. (2023) that length generalization depends on how positions are presented. These works change how text is encoded and ask what the model then does. We hold the encoding fixed instead: our spaced and digit controls give one token per character whether or not markers are present, so a marker's remaining effect cannot be a change in how characters are grouped into tokens. That separation is what lets us report a re-tokenization effect and a second effect that survives it, and it is why we can attribute the post-marker failure to the boundary rather than to the tokenizer.

Learned segmentation. Tokenizer-free models learn where to place boundaries. H-Net learns content-dependent chunks over bytes end to end (Hwang et al., 2026); H-Net++ adds a small cross-chunk attention module because chunking alone ignored dependencies across chunks (Zakershahrak & Ghodratnama, 2025); and MrT5 learns to delete byte tokens partway through the encoder, merging their information into the tokens that remain (Kallini et al., 2025). Our markers are the opposite case: fixed, content-free boundaries inserted at inference time into a model never trained to use them. The blind spot raises a question for learned

chunkers too, namely whether the first position after a learned boundary also loses access to what came before it; the H-Net++ design choice suggests that dependencies across boundaries need explicit support there as well.

Attention sinks and separator tokens. Language models concentrate attention on a few tokens, most famously the first token, which act as sinks (Xiao et al., 2024; Gu et al., 2025; Zhang et al., 2025); Su et al. (2026) survey uses and interpretations. SepLLM (Chen et al., 2025) shows that separator tokens can summarize the segment before them, SinkLoRA (Zhang, 2024) exploits sinks during fine-tuning, and punctuation tokens have been used as boundary anchors for trained sparse attention (Qiu et al., 2026). Oh et al. (2024) find that tabular structure in a prompt improves factual data interactions. Those separators are learned or exploited during training, and the question is what the model has come to do with them. Ours are inserted at inference time, and no model is trained or fine-tuned for this use; the marker character is of course familiar from pretraining, but nothing ties it to these tasks. That is why the marker can be content-free: it carries no task information, so any effect must come from the boundary itself rather than from what the token means. Our head ablations further show that the benefit is not carried by the heads that attend to markers most, which an attention-sink account would predict.

Attention heads. Induction heads implement in-context copying (Olsson et al., 2022), and causal head gating assigns roles to heads by intervention (Nam et al., 2025). We characterize the marker-attending heads and block them from the markers.

Long-sequence failure. Models use the middle of long contexts poorly (Liu et al., 2024), make sparse but systematic errors on flip-flop state tracking (Liu et al., 2023), and fail long-horizon tasks for reasons that compound (Wang et al., 2026; Pushkin & Abbe, 2026; Anand et al., 2026). Hou et al. (2026) quantify the accuracy cliff on repetitive deterministic tasks of the kind we use. Architectural remedies target long-sequence degradation directly; SWAN, for example, interleaves global attention layers without positional encoding with sliding-window rotary layers and extrapolates well beyond its training length (Puvvada et al., 2025). Relative position encodings such as ALiBi (Press et al., 2022) and FIRE (Li et al., 2024) likewise improve generalization to longer inputs. Markers act at the prompt, on unmodified models. The tokenization-independent benefit we find is a long-sequence effect of this kind.

Decomposition and memory. Least-to-most prompting (Zhou et al., 2023), program-of-thoughts (Chen et al., 2023), and scratchpads (Nye et al., 2021) decompose computation; Transformer-XL (Dai et al., 2019), recurrent memory (Bulatov et al., 2022), landmark tokens (Mohtashami & Jaggi, 2023), and learned context management (Kontonis et al., 2026) build segmentation or memory into the model. Content-free markers sit at the cheap end of this spectrum.

Benchmarks. Our tasks are generated fresh for every trial, following the contamination-resistant approach of BeyondBench (Srivastava et al., 2026). External checks use ProcBench (Fujisawa et al., 2024), the copying task of Jelassi et al. (2024), and CLRS-Text (Markeeva et al., 2026).

3 Setup

3.1 Tasks and formats

Every task maps an input string of length N to an output string of the same length under a deterministic rule stated fully in the prompt (Table 1). In *position-local* tasks, output y_i depends only on input x_i. In *one-step dependency* tasks, y_i also depends on x_{i-1}; in the *windowed* task, on x_{i-2}, x_{i-1}, x_i. For dependency tasks, the first position is copied as-is. Inputs are sampled uniformly with a deterministic seed per (task, length, spacing, alphabet size, trial), so every model sees the same inputs. `copy_previous` requires no computation, only a one-step lookback, so any harm from markers cannot be blamed on arithmetic; `sliding_window_max` uses a three-symbol window, so its output stays informative at every length.

Table 1: Tasks. A = alphabet size, M = modulus. Lengths $N \in \{4, 8, 16, 32, 64\}$. Position-local tasks are run with letters, spaced letters, and digits; dependency tasks use digits.

Task	Tier	Rule	Axis
`cyclic_shift`	position-local	next symbol in the alphabet, wrapping	$A \in \{2, 4, 8\}$
`random_permutation`	position-local	fixed arbitrary substitution (a derangement)	$A \in \{2, 4, 8\}$
`conditional_shift`	position-local	symbols at even alphabet positions forward, odd back	$A \in \{2, 4, 8\}$
`copy_previous`	one-step	$y_i = x_{i-1}$	binary
`running_xor`	one-step	$y_i = x_i \oplus x_{i-1}$	binary
`running_sum_local`	one-step	$y_i = (x_i + x_{i-1}) \bmod M$	$M \in \{2, 4, 8\}$
`sliding_window_max`	windowed	$y_i = \max(x_{i-2}, x_{i-1}, x_i)$	$A \in \{4, 8\}$

The position-local tasks are posed in three formats. In the *letter* format (the default), letters are written contiguously, and tokenizers merge them: on every model we test, 48 random letters encode to 23 to 32 tokens without markers and to one token per letter with a marker after each letter. In the *spaced* format, the same letters are separated by single spaces and the model is asked to answer in the same format; the 48 letters encode to exactly 48 tokens on every model whether or not markers are present, and each marker adds one token of its own. In the *digit* format, the letters are relabeled as digits ($A \to 0$, $B \to 1$, ...), which every tokenizer we test encodes one per character. Both controls are exact transformations of the letter tasks: same seeds, same positions, same rule and prompt wording (verified in every cell). The dependency tasks use digits throughout. One caveat on `conditional_shift`: its rule depends on a symbol's position in the alphabet, which for digits equals the digit itself, so its digit version is easier.

3.2 The marker intervention

In the marker condition ("Bounds"), a `|` is inserted after every k input characters, with $k \in \{1, 2, 4, 8, 16\}$; only $k < N$ is used. The expected output is the ground truth computed on the continuous, marker-free sequence, with markers spliced in at the same positions. The prompt states the rule, then instructs, for example: *"The input contains vertical bars (|). These are structural boundaries. DO NOT transform them. Copy them exactly to the output."* (the verb matches the task, e.g. "shift" for `cyclic_shift`). The no-marker baseline ("Base") uses the identical prompt, including that instruction, so that the only difference between conditions is whether markers appear in the input. That instruction is odd for an input with no bars in it, and Section 7.1 measures what it costs: dropping it raises no-marker accuracy by 0.02 to 0.04 at Qwen2.5-32B, so the marker effects we report are overstated by about that much against a baseline whose prompt never mentions bars. Example prompts are in Appendix A. Runs also included $k = 3$, which we exclude to keep the spacing grid at powers of two.

3.3 Models and decoding

Open models are Qwen2.5-7B/14B/32B-Instruct (Qwen Team, 2024), Mistral-7B-Instruct-v0.3 (Jiang et al., 2023), Phi-3.5-mini-instruct (Abdin et al., 2024), Qwen3.8-27B (Qwen Team, 2026), and Gemma-4-31B-it (Gemma Team, 2026), run locally in bfloat16 with each model's chat template and no system message beyond the template's own default (thinking disabled for Qwen3.8 and Gemma 4) and greedy decoding. Runs use PyTorch 2.6.0 (CUDA 12.4) and Transformers 5.12.1 with the eager attention implementation, on NVIDIA A100 80GB GPUs (Titan RTX for some smaller-model runs); Appendix B lists the exact checkpoint revisions. Qwen3.8-27B is a hybrid architecture: 48 of its 64 layers use Gated DeltaNet linear attention (Yang et al., 2025) and only 16 use softmax attention; Gemma-4-31B-it uses softmax attention in every layer (50 sliding-window and 10 global layers). The generation budget is $1.5N + 20$ new tokens, or $2N + 20$ at $k = 1$, where a correct marked answer is $2N - 1$ tokens. Predictions are clipped to the answer's length before scoring, so a budget can matter only by cutting an answer short; among the 86,400 generations whose raw output we stored, this happened in at most 0.6% of any condition class, at similar rates with and without markers,

and in at most 0.04% at the four stronger models (Appendix B).[1] The closed model is Gemini 3.5 Flash (Google, 2026), queried through its API between 2026-08-11T23:28 and 23:45 UTC with the same prompts and seeds, at temperature 0 with thinking disabled (`thinking_budget` 0), which is the closest available match to the open models' greedy decoding. We pinned the versioned model name rather than the `-latest` alias, and every response reported `gemini-3.5-flash` as the model actually served; the runs predate the October 2026 change that routes `gemini-3.5-flash` requests to `gemini-3.6-flash`. The resolved model name and timestamp are stored with every generation. It was run on the letter and dependency tasks but not on the two controls. Qwen2.5-32B is our primary model for head-level experiments.

3.4 Metrics and statistics

Character accuracy is the fraction of output positions whose character matches the ground truth, after stripping markers and spaces from both sides and clipping the prediction to the ground-truth length. We avoid the name "token accuracy" deliberately: the unit is a character of the answer, not a tokenizer token. Marker placement is not part of this metric; Appendix L reports exact-sequence match and marker-placement compliance separately.

Pooled effects. Only spacings $k < N$ are run, so marker runs are concentrated at longer lengths than no-marker runs, and accuracy falls with length. A pooled effect therefore compares the mean character accuracy of the marker runs with the no-marker mean reweighted to the same length distribution, $\Delta = \overline{\text{acc}}_{\text{Bounds}} - \sum_N p_N \, \overline{\text{acc}}_{\text{Base},N}$, where p_N is the share of marker runs at length N; it pools $N \leq 64$, $k \in \{1, 2, 4, 8, 16\}$, and alphabet or modulus values, and z is Δ over its standard error. Because the controls leave strong models near ceiling, we also report the *share of remaining errors removed*: the mean Δ over a model's tasks divided by one minus the mean length-matched no-marker accuracy. Open models use 20 to 80 trials per cell (Appendix B); Gemini uses 3. We call a pooled cell significant at $|z| \geq 3.51$, the Bonferroni-corrected two-sided $p < 0.05$ across all 112 pooled cells we test (the 70 in the overview of Section 6 and 42 for the two controls).

What the uncertainty is over. Every generation uses its own input, drawn from a seed determined by (task, length, spacing, alphabet, trial). Because the seed includes the spacing, a marker run and the no-marker run it is compared with see *different* random inputs from the same distribution: comparisons are between conditions, not paired by input. The unit of uncertainty is therefore the generation, and standard errors are the usual $s/\sqrt{n}$ over generations, combined across the two conditions as $\text{SE}(\Delta) = \sqrt{\text{SE}^2_{\text{Bounds}} + \sum_N p_N^2 \, \text{SE}^2_{\text{Base},N}}$, with $z = \Delta/\text{SE}(\Delta)$. A pooled cell mixes lengths, spacings, and alphabet values of differing difficulty, so Appendix C repeats every pooled cell with two bootstraps. Resampling generations supports the pooled effects (the Bonferroni verdict changes in 1 of 91 cells); the more conservative bootstrap over whole experimental configurations is much wider, leaving 17 of 91 cells above the threshold instead of 62, largely because the grid deliberately spans conditions where the effect is absent and because each cell contains few configurations. We therefore read pooled significance as conditional on the grid we ran, and rely on the length-resolved and position-resolved analyses to establish the structure of the effects.

Redundant configurations. At $A = 2$ the three position-local rules coincide: the only derangement of two symbols, the cyclic successor, and the conditional shift all flip the bit. At $M = 2$, `running_sum_local` computes `running_xor`. These cells still test different *instructions* for the same function, but they are not independent evidence across tasks; Appendix C recomputes every pooled effect with them dropped.

Per-position accuracy. For a marker run with spacing k, output position i (0-indexed, markers stripped) is *post-marker* if $i > 0$ and $i \bmod k = 0$, and *within-segment* otherwise. Each class is compared with the no-marker run's accuracy at the same position indices, which controls for accuracy varying along the sequence. Per-position results pool $N \leq 64$, $k \in \{2, 4, 8, 16\}$, and alphabet or modulus values; $k = 1$ has no within-

[1] The $k = 1$ cells at $N \geq 64$ were first generated under the smaller budget, which cut them off, and then regenerated; with greedy decoding the new outputs extend the old ones, except for Mistral, Phi, and Qwen2.5-7B on two tasks, whose original runs used a different GPU model and diverge after a shared prefix with the same accuracy over it.

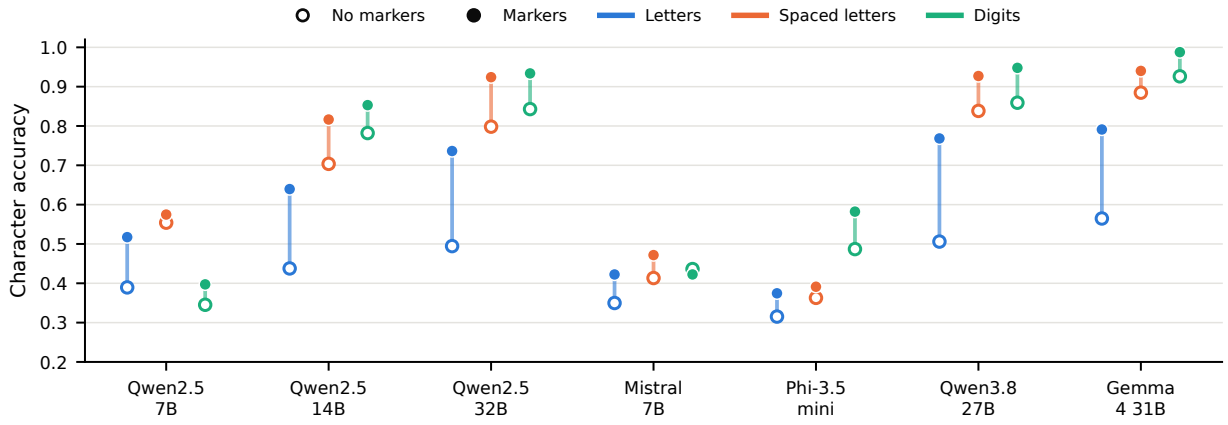


Figure 2: Character accuracy without markers (hollow) and with markers (filled) on the three position-local tasks in three formats: contiguous letters (blue), spaced letters (orange), and digits (green). Mean over tasks; $N \leq 64$, $k \in \{1, 2, 4, 8, 16\}$; no-marker values are length-matched to the marker runs (Section 3); whiskers (95% intervals) are mostly hidden by the markers. At the Qwen and Gemma models, spacing the letters without markers beats markers on contiguous letters. With tokenization fixed, markers still help the stronger models.

segment positions, so per-position claims concern spacings of two or more. The unit for uncertainty is the generation, and intervals are 95%.

4 On letter strings, markers mostly re-tokenize

Figure 2 compares the three formats. On contiguous letters, markers raise accuracy at every model: by +0.26 at Qwen3.8-27B, +0.24 at Qwen2.5-32B, +0.23 at Gemma-4-31B, +0.20 at Qwen2.5-14B, +0.13 at Qwen2.5-7B, +0.07 at Mistral-7B, +0.06 at Phi-3.5-mini, and +0.22 on Gemini (all 24 cells positive, 22 significant). But at the five Qwen and Gemma models, simply spacing the letters, with no markers, helps more: +0.33 at Qwen3.8, +0.32 at Gemma 4, +0.30 at Qwen2.5-32B, +0.27 at Qwen2.5-14B, and +0.16 at Qwen2.5-7B. At these models, spaced letters without markers are more accurate than contiguous letters with markers (for example, 0.80 against 0.74 at Qwen2.5-32B and 0.89 against 0.79 at Gemma 4); at Mistral (+0.06 from spacing) and Phi (+0.05), spacing and markers help about equally. Without markers, the letter tasks are hard for these models largely because of how letters are tokenized: averaged over lengths, the four stronger models solve the digit versions at 0.82 to 0.94, against 0.47 to 0.61 with contiguous letters.

The per-position view (Figure 3) shows how. On contiguous letters, the position right after a marker beats the no-marker run at the same positions in all 24 task–model cells, with per-model means from +0.07 (Phi) to +0.34 (Qwen2.5-32B and Qwen3.8), and it beats the within-segment positions of the same marker runs in 22 of 24 cells. A marker gives the following letter its own token. With spaced letters or digits, where every symbol is already its own token, that advantage is gone: the post-marker position beats the within-segment positions in only 10 of 21 cells in each control, and the two classes are within 0.04 of each other at every model, with the post-marker position slightly the lower of the two at Qwen2.5-7B (Appendix Table 9). The spacing profile tells the same story (Figure 4b, next section): on contiguous letters at $N = 64$, the gain is largest with a marker after every letter (+0.47 at the four stronger models) and falls steadily as markers get sparser (+0.09 at $k = 16$), as expected if markers help by splitting letter runs into shorter tokens. And the letter gain is present even on four-letter strings (+0.15), where the controls show no marker effect at all.

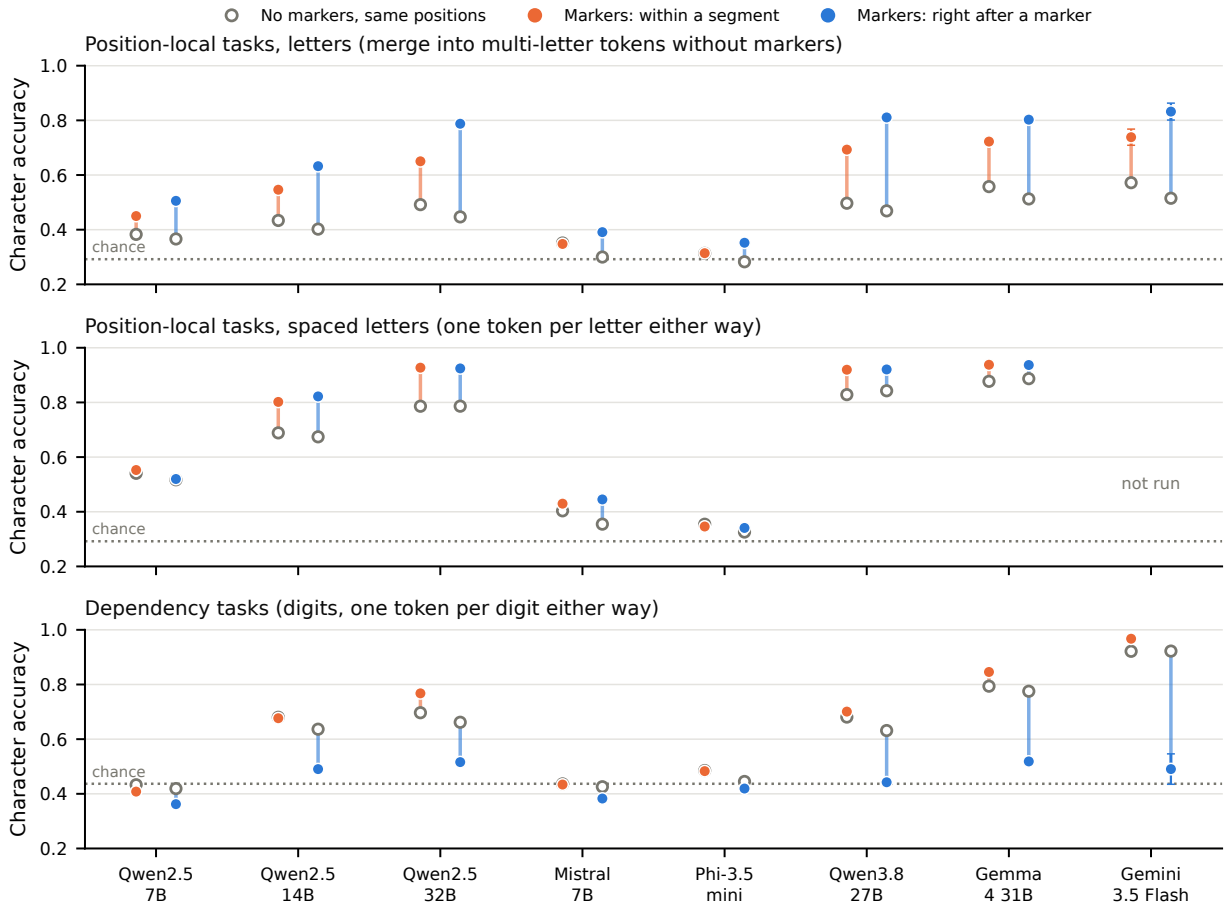


Figure 3: Per-position accuracy, every model. For each model: accuracy within a segment (orange) and at the position right after a marker (blue), each next to the no-marker run's accuracy at the same position indices (hollow gray). Mean over the tasks in each block; $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals; dotted line: chance (always guessing the most frequent output symbol), averaged over the tasks. Top: on contiguous letters, the post-marker position gains the most. Middle: with spaced letters it is no better than the rest of its segment. Bottom: on dependency tasks it loses at every model (Section 6). Gemini was not run on spaced letters.

5 With one token per character, markers still help strong models on long sequences

Holding each character to one token does not hold the whole prompt fixed: markers still add tokens, lengthen the sequence, and shift every later position. What the controls establish is narrower, that the gain is not an artifact of merging characters into multi-character tokens; we call it *tokenization-independent* in that sense throughout. With tokenization fixed, markers still help the stronger models. With spaced letters, the gain is +0.13 at Qwen2.5-32B, +0.11 at Qwen2.5-14B, +0.09 at Qwen3.8, and +0.06 at Gemma 4, removing 62%, 38%, 55%, and 48% of their remaining errors; with digits it is +0.06 to +0.09, removing 33% (Qwen2.5-14B) to 83% (Gemma 4). These effects are significant in all 12 spaced-letter cells and 11 of 12 digit cells at these four models. At the three smallest models the gains are smaller and inconsistent: with spaced letters, +0.02 (Qwen2.5-7B), +0.06 (Mistral), and +0.03 (Phi); with digits, +0.05, −0.01, and +0.10; significant in 7 of these 18 cells.

This benefit is a long-sequence effect (Figure 4a). Up to $N = 16$, the four stronger models are near ceiling in both controls (0.92 to 0.96 without markers) and markers change nothing. At $N = 32$, accuracy without

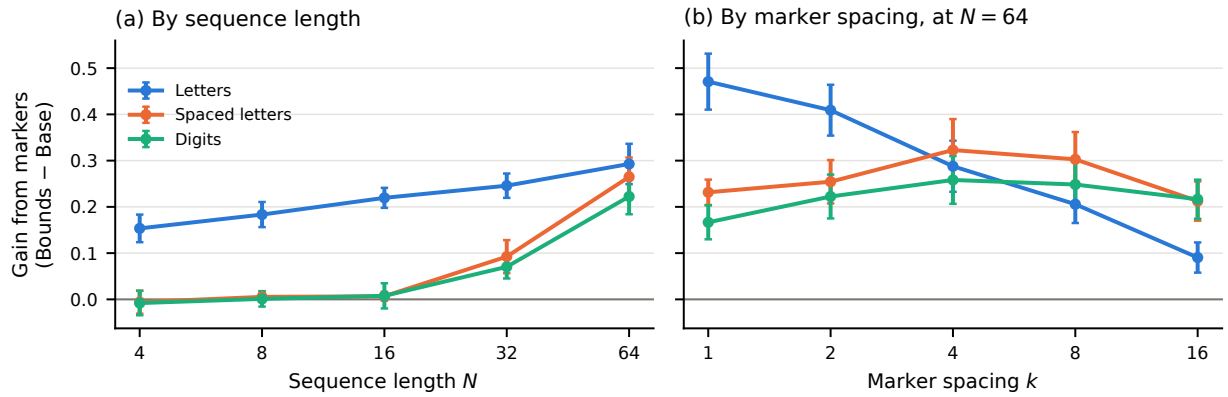


Figure 4: The marker effect (Bounds − Base) at the four stronger models (Qwen2.5-14B and 32B, Qwen3.8-27B, Gemma-4-31B), mean over the three position-local tasks. (a) By sequence length, pooled over spacing. (b) By marker spacing at $N = 64$. Whiskers are 95% intervals over the 12 model–task pairs. The two tokenization controls (spaced letters, digits) agree: no effect up to $N = 16$, a large effect at $N = 64$, and little dependence on spacing. Contiguous letters show a different signature: a gain at every length that is largest for the densest markers.

markers starts to fall and markers add +0.07 to +0.09; at $N = 64$, where accuracy without markers is 0.59 (spaced letters) and 0.68 (digits), markers add +0.27 and +0.22. In the strongest case, Qwen2.5-32B on `random_permutation` with eight digits at $N = 64$, accuracy rises from 0.41 to between 0.87 and 0.99 with any spacing from every 2 to every 16 characters. The gain depends little on spacing (Figure 4b): at $N = 64$ it is +0.21 to +0.32 for spaced letters and +0.17 to +0.26 for digits across all spacings, slightly weaker at the extremes (a marker after every character, which doubles the length of the output, or after every 16).

The gain is spread over the whole segment rather than concentrated after the marker (Figure 3, middle). A similar tokenization-independent gain appears inside segments on the dependency tasks, which use digits: within-segment positions gain +0.07 at Qwen2.5-32B, +0.05 at Gemma 4 and Gemini, and +0.02 at Qwen3.8 over the no-marker run at the same positions, though not at Qwen2.5-14B. Under edit-distance alignment these within-segment gains shrink but stay positive at the stronger models (for example, Qwen2.5-32B: +0.11 to +0.03 on `running_xor` and +0.11 to +0.05 on `running_sum_local`; Qwen3.8: +0.13 to +0.06 and +0.14 to +0.09), so part of what markers do inside segments is keep the model from slipping out of alignment. The gain also extends beyond the lengths of our main grid: at Qwen2.5-32B, on `random_permutation` with digits ($A = 8$, $k \in \{4, 16\}$), markers remove 89% to 91% of the remaining errors at $N = 64$, 78% to 81% at $N = 128$, and 56% to 57% at $N = 256$ (accuracy 0.18 without markers, 0.64 to 0.65 with), the ranges spanning `|`, commas, and line breaks as markers (Appendix Table 20). One reading is that a marker every few characters helps a strong model keep its place in a long sequence; Section 7 states that hypothesis precisely and tests the simplest version of it. The experiments in this section establish the effect, not its cause. Section 7 shows that the heads that attend most to markers carry little of this effect, and Qwen3.8-27B, whose layers are mostly linear attention, shows it comparably to the other strong models.

6 The position after a marker fails on dependencies that cross it

On the four dependency tasks, with a marker every two or more characters, the position right after a marker is below the no-marker run at the same positions in 29 of 31 task–model cells (Figure 3, bottom; these tasks use digits, so tokenization plays no role). The two exceptions are slight (+0.02 each, Qwen2.5-7B on `running_xor` and Phi on `sliding_window_max`); every cell at the five strongest models shows the loss. We call this the *blind spot* for short. The term describes the behavior, not a claim about what the model represents: accuracy near chance shows that the information before the marker is not used correctly here, not that it is inaccessible to the model. The loss grows with model strength: −0.03 to −0.06 at Phi, Mistral, and

Table 2: Instruction wording and the blind spot: per-position change from adding markers (marker run minus the no-marker run of the same wording, at the same positions). Original: the bars are "structural boundaries". Neutral: copy each bar through. Across: the bars are only visual separators and the rule applies across them. $N \leq 64$, $k \in \{2, 4, 8, 16\}$, 20 trials per cell.

		Post-marker			Within-segment		
Task	Model	Original	Neutral	Across	Original	Neutral	Across
`running_xor`	Qwen2.5 32B	−0.19	−0.17	−0.14	+0.11	+0.06	+0.09
`running_xor`	Qwen3.8 27B	−0.06	−0.07	−0.03	+0.13	+0.07	+0.13
`running_xor`	Gemma 4 31B	−0.26	−0.25	−0.15	+0.09	+0.07	+0.03
`running_sum_local`	Qwen2.5 32B	−0.16	−0.15	−0.14	+0.11	+0.07	+0.09
`running_sum_local`	Qwen3.8 27B	−0.06	−0.07	−0.04	+0.14	+0.13	+0.14
`running_sum_local`	Gemma 4 31B	−0.41	−0.39	−0.15	+0.11	+0.08	+0.07

Qwen2.5-7B; −0.15 at Qwen2.5-14B and 32B; −0.19 at Qwen3.8; −0.26 at Gemma 4; and −0.43 on Gemini, which is near ceiling without markers. At the stronger models the post-marker position is close to chance: on `running_xor` (chance 0.5) it is correct 0.51 to 0.54 of the time at Qwen2.5-14B and 32B, Qwen3.8, Gemma 4, and Gemini. The failure is confined to that position, with one qualification: on `sliding_window_max` the second position after a marker also needs input from before the marker, and we count it as within-segment, which makes the dependency-task estimates conservative for that class. With a marker every 16 characters, accuracy one position later is back at 0.85 (Qwen2.5-32B), 0.70 (Qwen3.8), 0.82 (Gemma 4), and 1.00 (Gemini). A marker after every character ($k = 1$) is different: there every position follows a marker, and on `copy_previous` at $N = 32$ Qwen2.5-32B is more accurate with such markers (0.88) than without (0.83), plausibly because a fully regular pattern makes the lookback fixed (Section 7).

Not an artifact of scoring. Character accuracy compares characters index by index, so a dropped or inserted character misaligns everything after it. Re-scoring every run with an edit-distance alignment, which repairs such slips instead of counting the rest of the output as wrong, leaves the post-marker loss in place at every strong model and task (Appendix Table 11). At Qwen2.5-32B it is −0.24 on `running_xor` and −0.22 on `running_sum_local` under alignment, against −0.19 and −0.16 index by index.

Not a matter of wording. Our instruction calls the bars "structural boundaries", which might invite the model to treat each segment as a fresh start. We reran `running_xor` and `running_sum_local` at Qwen2.5-32B, Qwen3.8, and Gemma 4 with two other wordings (Table 2): a *neutral* one that only asks for each bar to be copied through, and an *across* one stating that the bars are only visual separators and that the digit right after a bar uses the digit right before it as its previous input. The neutral wording changes the post-marker loss by at most 0.02. The across wording narrows it, most at Gemma 4 (from −0.41 to −0.15 on `running_sum_local` and from −0.26 to −0.15 on `running_xor`) and least at Qwen2.5-32B (−0.16 to −0.14 and −0.19 to −0.14), but in every cell the post-marker position stays below the no-marker run, and at Qwen2.5-32B it remains near chance on `running_xor` (0.58; chance 0.5). The framing does not create the blind spot, and an instruction that names the dependency explicitly repairs it only in part.

Other delimiters and longer sequences. At Qwen2.5-32B we replaced `|` with a comma or a line break, each with a constraint sentence naming it, and extended `running_xor` and `running_sum_local` (modulus 4) to $N = 128$ and 256 ($k \in \{4, 16\}$, 20 trials per cell; Appendix Table 20). With every delimiter and at every length, the post-marker position is at chance: 0.48 to 0.52 on `running_xor` (chance 0.5) and 0.21 to 0.26 on `running_sum_local` (chance 0.25). Within-segment positions gain +0.16 to +0.26 and +0.09 to +0.19 over the no-marker run at the same positions. Measured against the no-marker run, the post-marker loss shrinks with length (on `running_xor`, −0.11 to −0.14 at $N = 64$, −0.03 to −0.05 at 128, and +0.03 to +0.05 at 256), but only because the no-marker run itself decays toward chance on long sequences; the post-marker position does not improve. The blind spot is a property of the boundary, not of the symbol. The line break gives the largest pooled gain in all six dependency cells, by up to 0.08.

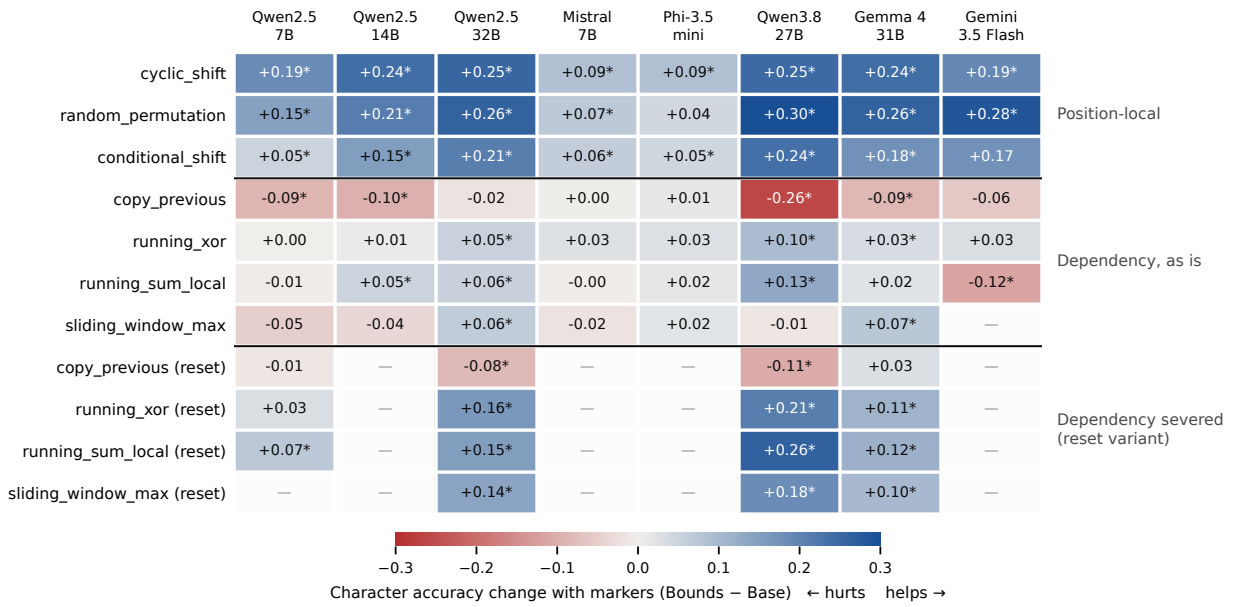


Figure 5: Pooled change in token accuracy when markers are added (Bounds − Base), contiguous-letter position-local tasks and digit dependency tasks, every model. Pooled over $N \in \{4, \ldots, 64\}$, $k \in \{1, 2, 4, 8, 16\}$, and alphabet or modulus values, with the no-marker mean length-matched to the marker runs. *: $|z| \geq 3.51$ (Bonferroni across the 112 pooled cells we test). Blue: markers help; red: markers hurt; —: not run. Bottom: dependency tasks rebuilt so the position after each marker needs nothing before it (Section 6.1).

Input markers alone are enough to cause it. Markers in the input with an output asked for without them leave the post-marker position at chance on `running_xor` (Section 7.1), so the failure follows the segmentation of the input, not the request to emit markers.

Prompt delivery. Qwen2.5's chat template inserts a system prompt of its own when none is given, so every run above carries one. Delivering the same prompts three other ways at Qwen2.5-32B leaves both effects in place (Appendix Table 23): with an empty system message, a task-oriented system message, or no chat template at all, the gain on `random_permutation` with digits is +0.36 to +0.39 against +0.37 by default, and on `running_xor` the post-marker position stays at chance (0.51 to 0.53, chance 0.5) while within-segment positions gain +0.14 to +0.18. The no-template condition is the closest an instruction-tuned checkpoint comes to base-model use, so this is weak evidence that neither effect depends on instruction tuning; base checkpoints themselves we did not test.

Larger state spaces. The dependency tasks above use at most eight symbols. We reran `running_sum_local` at Qwen2.5-32B with modulus 8, 16, and 32, writing the symbols space-separated so that each is one token at every modulus (Appendix Table 22). The post-marker position sits at chance at every modulus: 0.13 against a chance level of 0.125 at $M = 8$, 0.054 against 0.062 at $M = 16$, and 0.043 against 0.031 at $M = 32$, while within-segment positions gain +0.03 to +0.08 over the no-marker run at the same positions. The blind spot is therefore not an artifact of a small alphabet. One caveat bounds the largest case: without markers the model reaches only 0.075 at $M = 32$ against a chance level of 0.031, so that column is compressed toward the floor.

Sampling. All runs above decode greedily. Repeating three tasks at Qwen2.5-32B with sampling ($N \in \{32, 64\}$, $k \in \{4, 16\}$; temperature 0.7 with top-p 0.8, and temperature 1.0 with top-p 0.95; one sample per prompt) changes little (Appendix Table 21). The post-marker position stays at chance, 0.51 to 0.52 on `running_xor` and 0.26 to 0.27 on `running_sum_local` under all three decoding regimes, and the gain on `random_permutation` with digits is +0.37 greedy, +0.38 at temperature 0.7, and +0.42 at temperature 1.0.

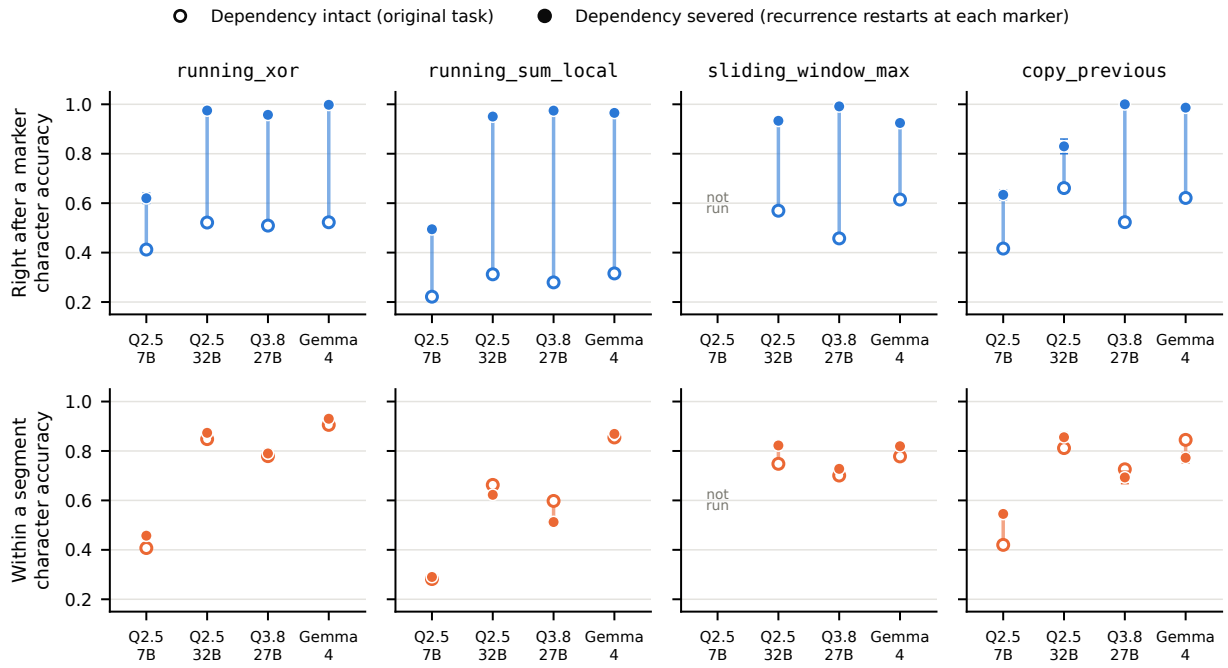


Figure 6: Reset variants, per position. Top row: accuracy at the position right after a marker, with the dependency intact (hollow) and with the recurrence restarting at each marker (filled). Bottom row: the same for within-segment positions. Models: Qwen2.5-7B and 32B, Qwen3.8-27B, Gemma-4-31B; `sliding_window_max` was not run in reset form at 7B. $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals.

Pooled effects mix the two. Averaged over all positions (Figure 5), a dependency task's response to markers combines this post-marker loss, on about a fraction $1/k$ of positions, with the within-segment gain of Section 5 on the rest, and the pooled sign varies accordingly: of 31 dependency cells, 8 are significantly positive (all on `running_xor`, `running_sum_local`, or `sliding_window_max` at the stronger models) and 5 significantly negative (four on `copy_previous`). Averaged over tasks, the effect ranges from -0.05 (Gemini) and -0.04 (Qwen2.5-7B) to $+0.04$ (Qwen2.5-32B). `copy_previous`, which needs no computation, is negative at six of eight models, significantly at four, including where the model is near ceiling without markers (Qwen3.8: -0.26). The pattern with length is consistent with this mixture: short sequences admit only dense spacings ($k < N$), so a larger share of their positions sit right after a marker, while long sequences add sparse spacings and are where the within-segment gain is large. At Qwen2.5-32B, `running_xor` goes from -0.11 at $N = 4$ to $+0.13$ at $N = 64$, and `running_sum_local` from -0.05 to $+0.14$; Gemma 4's `running_sum_local` goes from -0.10 at $N = 8$ to $+0.19$ at $N = 64$. Qwen3.8 gains most on balance on `running_xor` and `running_sum_local` ($+0.10$ and $+0.13$): its post-marker positions fail as at every model, but its within-segment positions on those two tasks gain $+0.13$ and $+0.14$.

6.1 Redefining the post-marker position

If the post-marker position fails because it needs the input before the marker, removing that need should repair exactly that position and leave the others alone. We rebuild each dependency task as a *reset* variant: the position right after each marker is treated like the first position of the sequence, so the recurrence restarts at every marker. The prompt states the new rule with a worked example (Appendix A). Reset variants were run at Qwen2.5-7B and 32B, Qwen3.8-27B, and Gemma-4-31B.

Figure 6 confirms the prediction. On `running_xor`, `running_sum_local`, and `sliding_window_max` at the three larger models, the post-marker position rises from 0.28–0.61 to 0.92–1.00, while within-segment

positions change by −0.09 to +0.07. Qwen2.5-7B moves the same way with smaller gains (`running_xor` 0.41 to 0.62; `running_sum_local` 0.22 to 0.49). Two caveats bound what this shows. The reset rule makes the post-marker position easier by design (on `running_xor` it becomes "copy this digit"), so the experiment localizes the harm rather than showing a new benefit. And the reset prompt alone shifts no-marker accuracy by −0.08 to +0.06, so pooled reset effects compare different baselines (Appendix Table 12); the per-position comparison uses marker runs only.

`copy_previous` behaves differently. Its post-marker position is also repaired (0.52–0.66 to 0.83–1.00), but its within-segment positions fall at Qwen3.8 (−0.03) and Gemma 4 (−0.07), and in the original task they are below the no-marker run at seven of eight models. Because the output is a shifted copy of the input, an off-by-one slip at a marker can carry through the rest of the segment; we read this as desynchronization, a failure distinct from the post-marker loss. Edit-distance alignment supports this reading: once slips are repaired, `copy_previous`'s within-segment loss shrinks from −0.24 to −0.05 at Qwen3.8 and from −0.10 to −0.05 at Qwen2.5-14B, while its post-marker loss remains (Appendix Table 11).

6.2 External tasks

On tasks we did not design (Appendix Tables 13 and 14, Figures 10 and 11), the pattern holds. On CLRS-Text's running-minimum algorithm (Markeeva et al., 2026), where every position depends on the whole prefix, markers hurt wherever the model can do the task: −0.20 ($z = 4.5$) at Qwen2.5-32B and −0.34 ($z = 7.3$) on Gemini. On ProcBench's per-character substitution task (Fujisawa et al., 2024), a position-local task, markers help (+0.10, $z = 6.7$ at Qwen2.5-7B; +0.20, $z = 10.8$ at 32B); its strings mix letters and digits, so part of this gain may be tokenization. On string copying (Jelassi et al., 2024), where accuracy without markers is above 0.97, the effect is null. Tagging each letter as a vowel or consonant from pretrained knowledge shows no benefit at Qwen2.5-32B (+0.01, $z = 0.5$); at 7B the baseline is at chance, so that cell is not interpretable.

7 What carries the effects? Marker-attending heads

This section has three kinds of content, and we keep them apart: what the attention patterns *show*, what the ablations *establish* by intervention, and what we *conjecture* about the mechanism. The interventions are narrower than the rest of the paper: they are run on Qwen2.5 only, on `random_permutation`, and they manipulate attention to markers rather than any other pathway. Read them as bounding what marker-attending heads contribute, not as a full account.

Conjecture: a segment-relative frame. To produce output position i, a model must find input position i (and, on dependency tasks, position $i-1$) among many near-identical tokens. We hypothesize that markers turn this global alignment into a local one: with a marker every k characters, the model can keep its place by the markers it has passed and its offset since the last one. This would explain a gain that appears only on long sequences, is spread over the whole segment, depends little on k, and shrinks when alignment slips are forgiven in scoring. The same local frame would explain the blind spot: if "the previous input" is looked up relative to the current segment, the first position of a segment finds a marker where its predecessor should be. This is a conjecture, not a result: the experiments below test only its simplest version, in which a few marker-attending heads carry the effects, and they rule that version out.

Observation: heads that attend to markers exist. In Qwen2.5, individual heads attend sharply from each position to the nearest marker (Appendix Figures 12 and 14). For the ten heads with the most attention to markers at Qwen2.5-32B, the mean percentile on a previous-token test is 95 and on an induction-head test 31 (Appendix Table 16); since heads selected for attending to an adjacent token will score high on a previous-token test, we read this only as "these are local heads". The total attention mass on markers rises with density, but so does the number of markers, so we do not claim markers are attention sinks in the sense of drawing more attention than content tokens.

Table 3: Head ablation at Qwen2.5-32B (`random_permutation`, $A = 8$, 20 trials per cell, pooled over $k \in \{2, 8\}$). "Top 10": the ten most marker-attending heads, chosen on held-out trials, are blocked from attending to markers (weights zeroed, rows renormalized). Controls: ten random heads blocked from markers, and the top ten blocked from as many random non-marker input tokens (sham). Share of gain removed is (Markers − blocked)/(Markers − No markers); "Controls" gives the larger of the two control shares.

				Markers, with heads blocked			Share of gain removed	
Format	N	No markers	Markers	Top 10	Random 10	Top 10, sham	Top 10	Controls
Letters	32	0.28	0.50	0.48	0.49	0.50	7%	$\leq 3\%$
Letters	64	0.19	0.45	0.40	0.45	0.44	19%	$\leq 3\%$
Spaced letters	32	0.70	0.99	0.98	0.98	0.98	1%	$\leq 2\%$
Spaced letters	64	0.38	0.90	0.83	0.88	0.87	13%	$\leq 5\%$
Digits	32	0.68	0.97	0.97	0.97	0.97	0%	$\leq 1\%$
Digits	64	0.41	0.96	0.95	0.97	0.96	1%	0%

Blocking those heads removes little of the gain. We block the ten most marker-attending heads from the markers during generation and compare with two controls (Table 3). On digits at Qwen2.5-32B, where markers raise accuracy from 0.41 to 0.96 at $N = 64$, blocking removes 1% of the gain (paired drop $+0.005 \pm 0.009$). At $N = 64$ it removes 13% on spaced letters ($+0.065 \pm 0.058$) and 19% on contiguous letters ($+0.049 \pm 0.030$), against at most 5% for either control; at $N = 32$ it removes 7% or less in every format. The heads that attend most to markers thus carry about none of the gain on digits, a modest and uncertain share on spaced letters, and up to a fifth on contiguous letters. An earlier ablation on contiguous letters (`cyclic_shift`, $A = 2$, $k = 2$) with heads re-identified per length removed 0% to 32% of the gain at Qwen2.5-32B and up to 74% at Qwen2.5-7B (Appendix Figure 15). At Qwen2.5-7B, marker gains in the new experiment are small and the drops from blocking are within the range of the controls (for example, $+0.03 \pm 0.05$ on spaced letters at $N = 32$, against $+0.03$ for random heads; Appendix Table 15). Blocking ten of the model's 2,560 heads is a narrow intervention, and the benefit may be distributed; what we can say is that it is not concentrated in the heads that attend to markers most.

Blocking many more heads removes part of it. To test whether the benefit is spread over heads, we repeated the ablation at $N = 64$ and $k = 8$ with the top 50 and top 200 marker-attending heads (2% and 8% of the model's heads), each with random-head and sham controls of the same size (Table 4). Blocking 50 heads removes 1% of the gain on spaced letters and 4% on digits. Blocking 200 removes 28% (paired drop $+0.155 \pm 0.072$) and 21% ($+0.115 \pm 0.083$), while 200 random heads blocked from the markers, or the top 200 blocked from as many random non-marker tokens, remove nothing (drops of -0.007 to -0.002). Because both controls of that size are null, the drop is specific to removing attention to markers rather than a general disruption. The benefit thus runs partly through attention to the markers, but thinly spread: even 200 heads carry about a quarter of it. (On spaced letters, the drops of about 0.04 for the 10-head conditions and the 50-head controls all come from one trial, the same one in each case; without it they are within 0.01 of zero.)

7.1 Input markers or output markers?

The intervention changes two things at once: it segments the input, and it asks for a segmented output. To separate them we ran a 2×2 at Qwen2.5-32B on `running_xor` and `random_permutation` with digits ($N \in \{32, 64\}$, $k \in \{4, 16\}$, 20 trials), varying whether the input carries markers and whether the output is asked to, plus a fifth condition whose prompt never mentions bars at all (Table 5). The three marker-bearing conditions are built from the same underlying string per trial, so they are paired.

What the conditions manipulate. The 2×2 varies what the prompt *asks for*. Whether the model complies is a separate matter, and here it often does not: asked to omit the markers from its output, it still emits them in 62% of `random_permutation` generations and 28% of `running_xor` ones; asked to insert markers into an output whose input has none, it produces the requested placement in 54% of `running_xor`

Table 4: Head-count dose response at Qwen2.5-32B (`random_permutation`, $A = 8$, $N = 64$, $k = 8$, 20 trials). Share of the marker gain removed, with the paired drop in accuracy and its 95% interval. Top n: the n most marker-attending heads blocked from markers; Random n: n random heads blocked from markers; Sham: the top n blocked from as many random non-marker tokens. Markers raise accuracy from 0.38 to 0.94 (spaced letters) and from 0.42 to 0.97 (digits). The 10-head rows use $k = 8$ only, so they differ slightly from Table 3, which pools $k \in \{2, 8\}$.

Format	Heads blocked	Top n	Random n	Sham
Spaced letters	10	8% ($+0.045 \pm 0.084$)	8% ($+0.042 \pm 0.083$)	7% ($+0.039 \pm 0.083$)
	50	1% ($+0.006 \pm 0.011$)	6% ($+0.037 \pm 0.084$)	7% ($+0.041 \pm 0.083$)
	200	28% ($+0.155 \pm 0.072$)	0% (-0.002 ± 0.009)	-1% (-0.004 ± 0.011)
Digits	10	1% ($+0.003 \pm 0.007$)	-2% (-0.012 ± 0.015)	1% ($+0.003 \pm 0.004$)
	50	4% ($+0.024 \pm 0.016$)	0% ($+0.001 \pm 0.004$)	-2% (-0.009 ± 0.013)
	200	21% ($+0.115 \pm 0.083$)	-1% (-0.006 ± 0.014)	-1% (-0.007 ± 0.011)

Table 5: Input markers, output markers, or both (Qwen2.5-32B; $N \in \{32, 64\}$, $k \in \{4, 16\}$ pooled, 20 trials per cell). Change is against "Marker sentence, no markers", the baseline used throughout the paper; $\pm$ is a 95% interval over generations. "Followed" is the share of generations whose output carried markers exactly as the prompt asked (none, or at the requested offsets); it is n/a where the main grid did not store the raw generations. Post-marker is accuracy at the position right after a marker, which exists only where the input carries markers (chance is 0.5 on `running_xor` and 0.125 on `random_permutation`).

		Markers					
Task	Condition	Input	Output	Accuracy	Change	Followed	Post-marke
`running_xor`	No marker sentence	no	no	0.70 ± 0.03	+0.04	100%	—
	Marker sentence, no markers	no	no	0.66 ± 0.03	—	n/a	—
	Markers in input only	yes	no	0.65 ± 0.03	−0.01	72%	0.52
	Markers in output only	no	yes	0.62 ± 0.02	−0.04	54%	—
	Markers in input and output	yes	yes	0.77 ± 0.02	+0.11	n/a	0.51
`random_permutation (digits)`	No marker sentence	no	no	0.57 ± 0.07	+0.02	100%	—
	Marker sentence, no markers	no	no	0.55 ± 0.06	—	100%	—
	Markers in input only	yes	no	0.76 ± 0.06	+0.22	38%	0.72
	Markers in output only	no	yes	0.56 ± 0.04	+0.01	0%	—
	Markers in input and output	yes	yes	0.92 ± 0.03	+0.37	94%	0.90

generations and in none of the `random_permutation` ones, where it emits on average 1.0 marker against 6.5 requested. The single-sided rows of Table 5 are therefore effects of the instruction, not of a realized format, and the output-only row on `random_permutation` tells us almost nothing about output markers because the model did not produce them.

What the experiment does show. First, the marker sentence itself is not free: with no markers anywhere, dropping it raises accuracy from 0.66 to 0.70 on `running_xor` and from 0.55 to 0.57 on `random_permutation`. Our baseline keeps that sentence, so the gains reported elsewhere in the paper are overstated by roughly 0.02 to 0.04; on these two cells the marker gain is +0.07 and +0.35 against the clean baseline rather than +0.11 and +0.37.

Second, asking for markers on both sides beats asking for either alone, by more than their sum. The interaction $A_{11} - A_{10} - A_{01} + A_{00}$ is +0.14 (95% interval $[0.06, 0.23]$) on `random_permutation` and +0.16 ($[0.11, 0.20]$) on `running_xor`, bootstrapped over the paired trials of the three marked conditions and, separately, over the unpaired baseline. Both intervals exclude zero.

Third, forcing the format settles it. We reran the same four cells with a logits processor that counts the answer characters as they are generated and either forces the marker token when one is due or removes every

Table 6: The same 2×2 with the output format enforced by a logits processor instead of requested in the prompt (Qwen2.5-32B; $N \in \{32, 64\}$, $k \in \{4, 16\}$ pooled, 20 trials per cell). "Format" is the share of generations whose output came out as intended; it is below 100% only where a marker was due inside a multi-character token. Change is against the no-marker cell, and the interaction is $A_{11} - A_{10} - A_{01} + A_{00}$ with a bootstrap interval. Post-marker is accuracy at the position right after a marker.

		Markers					
Task	Condition	Input	Output	Accuracy	Change	Format	Post-marker
`running_xor`	Neither side	no	no	0.66 ± 0.03	—	100%	—
	Input only	yes	no	0.63 ± 0.02	−0.03	100%	0.53
	Output only	no	yes	0.61 ± 0.02	−0.05	75%	—
	Both sides	yes	yes	0.77 ± 0.02	+0.10	100%	0.51
Interaction +0.19, 95% interval	[+0.15, +0.24]						
`random_permutation (digits)`	Neither side	no	no	0.55 ± 0.06	—	100%	—
	Input only	yes	no	0.40 ± 0.03	−0.15	100%	0.35
	Output only	no	yes	0.39 ± 0.03	−0.16	64%	—
	Both sides	yes	yes	0.94 ± 0.02	+0.39	100%	0.94
Interaction +0.70, 95% interval	[+0.61, +0.79]						

marker-bearing token from consideration, so the output format is certain rather than requested (Table 6). The constraint holds exactly in the no-marker, input-only, and both-sides cells; in the output-only cell it holds in 64% to 75% of generations, because a marker cannot be placed inside a multi-character token. Constraining does not itself move the results: the no-marker and both-sides cells reproduce their unconstrained counterparts to within 0.03.

With the format enforced, *neither side alone helps, and both hurt.* Markers in the input with an output genuinely free of them give −0.03 on `running_xor` and −0.15 on `random_permutation`; markers forced into the output of an unmarked input give −0.05 and −0.16. Only the combination helps, by +0.10 and +0.39. The interaction is +0.19 (95% interval $[0.15, 0.24]$) and +0.70 ($[0.61, 0.79]$). This also confirms the descriptive split above: the input-only cell under enforcement scores 0.40 on `random_permutation`, close to the 0.42 of the unenforced generations that happened to comply. A marker helps only when it appears on both sides, where it can align the output being written with the input being read; segmenting the input while writing an unsegmented output is worse than not segmenting at all.

Fourth, the blind spot follows the input markers alone. With markers in the input and an output enforced to carry none, the position right after a marker is at chance on `running_xor` (0.53, chance 0.5), as it is when markers appear on both sides (0.51). Segmenting the input is what breaks the cross-boundary dependency; emitting markers is not.

The same heads draw attention away from a needed input. On `copy_previous` at $N = 32$, a marker after every character keeps the marker pattern aligned with the one-step lookback, and attention from each output position to the input it needs is high (0.535). At $k = 2$, every other lookback crosses a marker; attention to the needed token falls to 0.101 and accuracy drops from 0.88 to 0.65, below the no-marker baseline of 0.83. At $k = 8$ both recover (Appendix Figure 12d). This is the attention-level counterpart of the blind spot.

A model with little softmax attention. Qwen3.8-27B, in which 48 of 64 layers use Gated DeltaNet linear attention (Yang et al., 2025) rather than softmax attention, shows the tokenization-independent gain and the blind spot comparably to the other strong models.

8 Practical considerations

Table 7 summarizes the guidance; the paragraphs below give the evidence.

Table 7: When to space characters, when to add markers, and when to avoid them. Based on the models and tasks tested here (Section 9 bounds the scope).

Situation	What to do
Characters run together and the tokenizer merges them (letter strings)	Put a space between characters first. At the Qwen and Gemma models this beats markers and costs fewer tokens
Sequence long enough that accuracy has started to fall (from about $N = 32$ here)	Add a marker every 4 to 16 characters. The gain depends little on the spacing, and sparse markers cost little
Short sequences, where the model is near ceiling	Markers add nothing; skip them
Each output depends only on the input at the same position	Markers are safe, and help most
An output depends on the input just before a marker	Do not place a marker there, or restate the rule so the position after each marker needs nothing before it
Latency matters	Sparse markers are faster than none, because they stop models from running past the required length

Separate characters first. For letter strings, spacing the characters beats inserting markers at every Qwen and Gemma model we test, matches markers at Mistral and Phi, and costs less: 48 spaced letters are 48 tokens, against 95 with a marker after every letter. On top of spacing, markers add a further gain at the stronger models on long sequences.

Use sparse markers on long sequences. The tokenization-independent gain appears from $N \approx 32$ and depends little on spacing, so a marker every 4 to 16 characters captures it at little cost. Markers also help models stop at the right length. Without markers, on contiguous-letter tasks at $N = 64$ to 128, 18% to 95% of outputs run past the required length (Qwen2.5-32B, Qwen3.8, Gemma 4), against 0% to 13% with a marker every 8 characters. Because generation speed is constant across conditions (14 to 15 tokens/s at Qwen2.5-32B), sparse markers therefore take less time than none: 0.66 times as long at $N = 64$ and 0.44 times at $N = 128$ (Appendix Table 27). True divide-and-conquer, with one call per segment, holds accuracy nearly flat (character accuracy 0.93 to 0.92 from $N = 8$ to 128) but costs about three times the wall-clock of the marker condition at $N = 64$ to 128 (Appendix Figure 19).

Avoid markers where an output needs the input before them. With a marker every two or more characters, the position after a marker fails wherever its output needs the input before the marker. Place markers only at points where nothing after them depends on what came before, or restart the computation at each marker. (A marker after every character behaves differently on `copy_previous`; see Section 6.)

What about the marker matters. Ablations on contiguous-letter `cyclic_shift` ($A = 2$, $k = 4$; Appendix Tables 17–19), where much of the effect is tokenization: replacing `|` with `;` or `#` keeps accuracy above the no-marker baseline at Qwen2.5-32B, though markers differ by up to 0.13 at $N = 128$; a reserved special token used as the marker performs far below the no-marker baseline; a frequent single-token marker beats a less frequent multi-token one with the same token budget; jittered or random placement falls below the baseline at most lengths at 7B but stays above it at 32B; and the benefit holds under formal and casual phrasings, but a terse cipher-style phrasing reverses it at 32B. These ablations cover one task and two models. Common delimiters differ from our marker in ways that may matter: whitespace between characters is our spacing control, which fixes tokenization but marks no segments; commas and semicolons, like `|`, are frequent single tokens, and `;` behaves much like `|` here; newlines also change the line structure the model sees. On the dependency tasks at Qwen2.5-32B, commas and line breaks produce the same blind spot and the same long-sequence gain as `|` (Section 6), so the advice on where to place markers applies to the delimiters we tested (bars, commas, and line breaks) and plausibly to other common ones. Markers also slightly reduce the share of well-formed outputs in six of seven tasks at Qwen2.5-32B (Appendix Table 26). The density that maximizes accuracy on contiguous letters falls with length (Appendix Figure 18), which we describe but do not fit (Appendix Table 24).

9 Limitations

- **Mechanism.** Blocking up to 200 marker-attending heads removes at most about a quarter of the tokenization-independent gain; we do not identify what carries the rest, and we did not test MLP or path-level interventions. Head experiments are on Qwen2.5 only and one task.

- **Controls not run everywhere.** The spaced-letter and digit controls were run on the seven open models, not on Gemini, whose tokenizer we cannot inspect.

- **Scoring.** Our main metric compares characters position by position, so a dropped or inserted character misaligns everything after it. An edit-distance-aligned re-scoring leaves the blind spot intact and attributes most of `copy_previous`'s within-segment loss to such slips (Appendix Table 11); a fully alignment-free metric would be more lenient still, especially on binary alphabets.

- **Floors and ceilings.** Raw differences in accuracy are not comparable across tasks with different baselines; the controls leave strong models near ceiling at short lengths, which is why we report shares of remaining errors. Gemini's dependency tasks start near ceiling, and some 7B, Mistral, and Phi cells start near chance.

- **Output-only enforcement.** In the output-only cell the forced format is realized in 64% to 75% of generations, because a marker cannot be inserted inside a multi-character token; the other cells are exact.

- **Baseline prompt.** The no-marker baseline keeps the sentence describing the markers, which costs it 0.02 to 0.04 at Qwen2.5-32B on the two tasks where we measured it (Section 7.1); reported gains are overstated by about that much, and we did not rerun the full grid with a clean baseline.

- **Reset design.** The reset rule makes the post-marker position easier by construction, and the reset prompt shifts no-marker accuracy by up to 0.08.

- **Spacing and pooling.** Per-position claims concern spacings of two or more ($k = 1$ has no within-segment positions, and behaves differently on `copy_previous`). Pooled effects depend on how lengths are weighted; we match the no-marker baseline to the marker runs' lengths (Section 3), which raises pooled effects relative to comparing raw means (by 0.03 in the median cell and at most 0.09).

- **Models and sample size.** All open models are instruction-tuned, and five are 2024-era. We do not test base checkpoints; stripping the chat template at Qwen2.5-32B, the closest available proxy, leaves both effects intact (Appendix Table 23), but that is not the same experiment. Reset variants at Qwen2.5-32B are single-batch (20 trials per cell); Gemini uses 3 trials per cell.

- **Length and decoding.** Core grids stop at $N = 64$; a follow-up on three tasks at Qwen2.5-32B reaches $N = 256$. Behavior over thousands of characters and at other models is untested. Main results use greedy decoding; sampling was checked on three tasks at Qwen2.5-32B only.

- **Marker variants.** Marker symbol, placement, and wording were ablated on one letter task at two models (Appendix I); commas and line breaks were tested on three tasks at Qwen2.5-32B only.

- **Position encoding.** A marker shifts the relative offset between an output position and the input it needs, so position-encoding schemes other than those of the models we test might change the blind spot; testing this requires models trained with them.

- **Task scope.** All tasks are synthetic character-level transformations with in-context rules, with alphabets or moduli of at most eight symbols and dependencies of one or two steps; a naturalistic task (vowel tagging) shows no benefit at 32B.

10 Conclusion

A content-free marker inserted every few characters does three separate things. On letter strings it splits multi-letter tokens, which accounts for much of its visible benefit, and spacing the characters does that job as well or better. Independently of tokenization, it helps strong models keep their place on long sequences, removing about half of their remaining errors; blocking large sets of marker-attending heads removes part of that gain, so marker-directed attention contributes, but the few heads that attend to markers most do not carry it and the rest of the mechanism is unresolved. And, with a marker every two or more characters, the position right after it fails whenever its output depends on the input before the marker, which is harmless for position-local rules and damaging wherever an output needs its predecessor. That failure is consistent with the model resolving "the previous input" within the current segment rather than across the boundary, though our experiments do not establish that mechanism. Separating the intervention's two halves, with the output format enforced, shows that marking either side alone hurts and only marking both helps, while the cross-boundary failure comes from the input side alone. Separate the characters, mark long sequences sparsely, and avoid placing markers where an output needs the input before them.

Reproducibility. [Code, prompts, raw generations, and analysis scripts: anonymized repository link.] All inputs are generated from deterministic seeds, and every prompt can be regenerated from (task, format, length, spacing, alphabet, trial).

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219, 2024.

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. arXiv preprint arXiv:2305.13707, 2023.

Avinash Anand, Mahisha Ramesh, Avni Mittal, Ashutosh Kumar, Rishitej Reddy Vyalla, Erik Cambria, Zhengkui Wang, Timothy Liu, Aik Beng Ng, Simon See, et al. The Periodic Table of LLM Reasoning: A Structured Survey of Reasoning Paradigms, Methods, and Failure Modes. arXiv preprint arXiv:2606.11470, 2026.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent Memory Transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2207.06881.

Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. SepLLM: Accelerate Large Language Models by Compressing One Segment into One Separator. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 9008–9028, 2025.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, 2023. arXiv:2211.12588.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. arXiv:1901.02860.

Shaoxiong Duan, Yining Shi, and Wei Xu. From Interpolation to Extrapolation: Complete Length Generalization for Arithmetic Transformers. arXiv preprint arXiv:2310.11984, 2023.

Lukas Edman, Helmut Schmid, and Alexander Fraser. CUTE: Measuring LLMs' understanding of their tokens. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

Ippei Fujisawa, Sensho Nobe, Hiroki Seto, Rina Onda, Yoshiaki Uchida, Hiroki Ikoma, Pei-Chun Chien, and Ryota Kanai. ProcBench: Benchmark for Multi-Step Reasoning and Following Procedure. arXiv preprint arXiv:2410.03117, 2024.

Gemma Team. Gemma 4. Model card, `https://huggingface.co/google/gemma-4-31B-it`, 2026.

Google. Gemini 3.5 Flash. Gemini API model `gemini-3.5-flash`, accessed August 2026, 2026.

Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When Attention Sink Emerges in Language Models: An Empirical View. In *International Conference on Learning Representations*, 2025.

Wanda Hou, Leon Zhou, Hong-Ye Hu, Yubei Chen, Yi-Zhuang You, and Xiao-Liang Qi. Quantitative Study of Sustained Focus in Large Language Models via Repetitive Deterministic Prediction Tasks. *Physical Review Research*, 8:043036, 2026. doi: 10.1103/fn1m-wn2b.

Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic Chunking for End-to-End Hierarchical Sequence Modeling. In *International Conference on Learning Representations*, 2026.

Kanishk Jain, Matthew Day, and Tankut Can. Emergent retokenization symmetry in large language models: phenomenology and applications. arXiv preprint arXiv:2606.15521, 2026.

Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. Repeat After Me: Transformers are Better than State Space Models at Copying. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.01032.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. arXiv preprint arXiv:2310.06825, 2023.

Julie Kallini, Shikhar Murty, Christopher D. Manning, Christopher Potts, and Róbert Csordás. MrT5: Dynamic Token Merging for Efficient Byte-level Language Models. In *International Conference on Learning Representations*, 2025.

Ayush Kaushal and Kyle Mahowald. What do tokens know about their characters and how do they know it? arXiv preprint arXiv:2206.02608, 2022.

Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyan Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. MEMENTO: Teaching LLMs to Manage Their Context. In *Conference on Language Modeling*, 2026.

Shanda Li et al. Functional Interpolation for Relative Positions Improves Long Context Transformers. In *International Conference on Learning Representations*, 2024.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Exposing Attention Glitches with Flip-Flop Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.00946.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. arXiv:2307.03172.

Larisa Markeeva, Sean Michael McLeish, Borja Ibarz, Wilfried Bounsi, Olga Kozlova, Alex Vitvitskyi, Charles Blundell, Tom Goldstein, Avi Schwarzschild, and Petar Veličković. The CLRS-Text Algorithmic Reasoning Language Benchmark. *Journal of Data-centric Machine Learning Research*, 2026.

Amirkeivan Mohtashami and Martin Jaggi. Landmark Attention: Random-Access Infinite Context Length for Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16300.

Andrew Nam, Henry Conklin, Yukang Yang, Thomas Griffiths, Jonathan Cohen, and Sarah-Jane Leslie. Causal Head Gating: A Framework for Interpreting Roles of Attention Heads in Transformers. In *Advances in Neural Information Processing Systems*, 2025.

Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Investigating the Limitations of Transformers with Simple Arithmetic Tasks. arXiv preprint arXiv:2102.13019, 2021.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv preprint arXiv:2112.00114, 2021.

Jio Oh, Geon Heo, Seungjun Oh, Hyunjin Kim, JinYeong Bak, Jindong Wang, Xing Xie, and Steven Euijong Whang. Talking with Tables for Better LLM Factual Data Interactions. arXiv preprint arXiv:2412.17189, 2024.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context Learning and Induction Heads. arXiv preprint arXiv:2209.11895, 2022.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems*, 2023.

Ofir Press, Noah A. Smith, and Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations*, 2022.

Denys Pushkin and Emmanuel Abbe. LEAD: Breaking the No-Recovery Bottleneck in Long-Horizon Reasoning. arXiv preprint arXiv:2603.06870, 2026.

Krishna C. Puvvada et al. SWAN: An Efficient and Scalable Approach for Long-Context Language Modeling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 2424–2438, 2025. doi: 10.18653/v1/2025.emnlp-main.123.

Junxiang Qiu et al. Punctuation-aware Hybrid Trainable Sparse Attention for Large Language Models. *arXiv preprint arXiv:2601.02819*, 2026.

Qwen Team. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115, 2024.

Qwen Team. Qwen3.8-27B. Model card, `https://huggingface.co/Qwen/Qwen3.8-27B`, 2026.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting. In *International Conference on Learning Representations*, 2024.

Ruoqi Shen, Sébastien Bubeck, Ronen Eldan, Yin Tat Lee, Yuanzhi Li, and Yi Zhang. Positional Description Matters for Transformers Arithmetic. arXiv preprint arXiv:2311.14737, 2023.

Gaurav Srivastava, Aafiya Hussain, Zhenyu Bi, Swastik Roy, Priya Pitre, Meng Lu, Morteza Ziyadi, and Xuan Wang. BeyondBench: Contamination-Resistant Evaluation of Reasoning in Language Models. In *International Conference on Learning Representations*, 2026.

Zunhai Su, Hengyuan Zhang, Wei Wu, Yifan Zhang, Yaxiu Liu, He Xiao, Qingyao Yang, Yuxuan Sun, Rui Yang, Chao Zhang, et al. Attention Sink in Transformers: A Survey on Utilization, Interpretation, and Mitigation. arXiv preprint arXiv:2604.10098, 2026.

Omri Uzan and Yuval Pinter. CharBench: Evaluating the Role of Tokenization in Character-Level Tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 2026.

Xinyu Jessica Wang, Haoyue Bai, Yiyou Sun, Haorui Wang, Shuibai Zhang, Wenjie Hu, Mya Schroder, Bilge Mutlu, Dawn Song, and Robert D Nowak. The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break. arXiv preprint arXiv:2604.11978, 2026.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

Mehrdad Zakershahrak and Samira Ghodratnama. H-Net++: Hierarchical Dynamic Chunking for Tokenizer-Free Language Modelling in Morphologically-Rich Languages. *arXiv preprint arXiv:2508.05628*, 2025.

Hengyu Zhang. SinkLoRA: Enhanced Efficiency and Chat Capabilities for Long-Context Large Language Models. arXiv preprint arXiv:2406.05678, 2024.

Stephen Zhang, Mustafa Khan, and Vardan Papyan. Attention Sinks: A 'Catch, Tag, Release' Mechanism for Embeddings. In *Advances in Neural Information Processing Systems*, 2025.

Brian Siyuan Zheng, Alisa Liu, Orevaoghene Ahia, Jonathan Hayase, Yejin Choi, and Noah A. Smith. Broken tokens? your language model can secretly handle non-canonical tokenizations. In *Advances in Neural Information Processing Systems*, 2025.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2205.10625.

A Prompts

`cyclic_shift` ($N = 8$, $A = 4$, $k = 4$).

```
You are a deterministic sequence transformer.
Task: Cyclic character shift.
Alphabet: A, B, C, D
Rule: Replace every character with the next one in the alphabet.
Mapping: D -> A, and A -> B.
Constraint: The input contains vertical bars (|). These are structural
boundaries. DO NOT shift them. Copy them exactly to the output.
No spaces, no explanations.

Input: CABA|CCDA
Output:                 (expected: DBCB|DDAB)
```

Spaced and digit formats. The spaced version of this cell shows the input as `C A B A | C C D A` and replaces "No spaces, no explanations." with "Write the output in the same format as the input: every character separated by a single space. No explanations."; everything else is unchanged. The digit version relabels A, B, C, D as $0, 1, 2, 3$ (input `2010|2230`), with the task line, rule, and mapping worded for digits ("Cyclic digit shift", "Replace every digit with the next one in the alphabet"). The same seed draws the same positions in all three formats.

`running_xor`, original and reset ($N = 8$, $k = 4$). Both prompts state the rule $y_i = x_i \oplus x_{i-1}$ with the first digit copied. The reset prompt adds: *"Immediately after each boundary, treat that position exactly like the first digit of the whole input—copy it as-is (nothing before a boundary counts as 'previous'), then resume the XOR rule from there until the next boundary,"* with the worked example `1011|0010` → `1110|0011`. For input `1101|1111`, the original expects `1011|0000`, while the reset variant expects `1011|1000`: only the digit after the marker differs.

`sliding_window_max` ($N = 8$, $k = 4$). The prompt states the rule with a worked example and adds: *"DO NOT include them [the bars] in the window or count them as digits."* For input `5170|1247` the expected output is `5577|7247`.

B Trials per cell

Table 8 gives trials per cell (one cell = task, model, length, density, and alphabet or modulus value) for the data pooled in Figures 2, 4, and 5. Gemini uses 3.

Table 8: Trials per cell, $N \leq 64$. Q2.5: Qwen2.5-Instruct; Q3.8: Qwen3.8-27B; Gemma 4: Gemma-4-31B-it.

Task	Q2.5-7B	Q2.5-14B	Q2.5-32B	Mistral	Phi	Q3.8	Gemma 4
`cyclic_shift`	20	20	20	40	20	20	20
`random_permutation`	40	40	40	40	20	20	20
`conditional_shift`	40	20	40	40	20	20	20
`copy_previous`	40	40	80	40	40	40	40
`running_xor`	40	40	40	40	20	40	40
`running_sum_local`	40	40	80	40	20	40	40
`sliding_window_max`	20	20	40	20	20	20	20
Reset variants	80	–	20	–	–	20	20
Spaced versions (each)	20	20	20	20	20	20	20
Digit versions (each)	20	20	20	20	20	20	20

Checkpoints. Hugging Face revisions: Qwen2.5-7B-Instruct `a09a35458c70`, Qwen2.5-14B-Instruct `cf98f3b3bbb4`, Qwen2.5-32B-Instruct `5ede1c97bbab`, Mistral-7B-Instruct-v0.3 `c170c708c41d`, Phi-3.5-mini-instruct `2fe192450127`, Qwen3.8-27B `37561816e66b`, Gemma-4-31B-it `842da3794eaa`.

Generation budget. The no-marker run and every spacing $k \geq 2$ share one budget, $1.5N+20$ new tokens; $k = 1$ uses $2N + 20$. To check that the budget does not drive any effect, we re-tokenized every stored raw output of the seven open models ($N \leq 64$; 86,400 generations, since older letter-task files store only the clipped answer) and counted those that used the whole budget while giving fewer than N answer characters on the first line. This is an upper bound on cut-off answers, because it also counts short answers followed by unrequested text. The share is 0.58% without markers, 0.49% at $k \geq 2$, and 0.31% at $k = 1$; at the four stronger models it is 0.04%, 0.02%, and 0%. Nearly all cases are Phi-3.5-mini on spaced letters (up to 19% of one cell without markers, against 14% with), and on the dependency tasks the share is 0.03% or less in every class.

C How the uncertainty was computed

Pooled effects use the length-matched estimator of Section 3, with analytic standard errors over generations. Two checks ask how much that choice matters (`scripts/analyze_statistics_robustness.py`; per-cell values in `results_clean/_summary/statistics_robustness.csv`). Both resample 2,000 times, over the 91 pooled task–model cells of the three position-local families and the dependency family.

Resampling generations reproduces the analytic result. Bootstrapping generations within each condition and length gives a median $|z|$ within 3% of the analytic value, and the Bonferroni verdict ($|z| \geq 3.51$) changes in 1 of 91 cells. The analytic standard errors are therefore not the weak point.

Resampling whole cells is far wider. Treating each (length, spacing, alphabet) cell as the unit instead, so that whole cells enter or leave a resample, widens the median interval from 0.046 to 0.155 and leaves 17 of 91 cells above the Bonferroni threshold instead of 62. Two things drive this, and neither is a defect in the data. First, a pooled cell deliberately spans lengths at which the effect is absent: at $N \leq 16$ these models are near ceiling and markers do nothing, which is itself Finding 2. Averaging those in and then asking how much the average moves when lengths are resampled mostly measures that by-design heterogeneity. Second, a pooled cell contains only 10 to 25 (length, spacing, alphabet) cells, and a bootstrap over so few clusters has little power.

Restricting to the length where the claim is actually made removes the first problem. At $N = 64$, all 24 spaced-letter and digit cells at the four stronger models have a positive effect, between +0.07 and +0.37 (mean +0.24); all 24 clear $|z| \geq 3.51$ analytically and under the generation bootstrap, 22 of 24 have a cell-level $|z| \geq 2$, and 4 of 24 clear 3.51 under the cell-level bootstrap with its 15 or so clusters (`results_clean/_summary/finding2_at_N64.csv`).

We therefore read a pooled z as a statement about the mean effect over the grid we ran, not as evidence that the effect is uniform across lengths and spacings, nor that it would survive a different choice of grid. The claims about structure rest on the length-resolved and position-resolved analyses (Figures 4 and 3), not on the pooled numbers.

Dropping the redundant configurations. At $A = 2$ the three position-local rules coincide and at $M = 2$ `running_sum_local` computes `running_xor` (Section 3). Recomputing all 91 pooled cells without those configurations moves the effect by 0.013 in the median cell and at most 0.054; the effect gets *larger* in 54 of 91 cells, because the two-symbol cells are the easiest and dilute the average. Three cells change sign, all with $|\Delta| \leq 0.009$ and none significant either way, and the Bonferroni verdict changes in 5 of 91 cells. Finding 2 is unchanged: 23 of its 24 cells remain significant.

D Per-model results

Figures 7 and 8 show accuracy by sequence length for each open model and task; Gemini is in Table 14. Pooled effects and their significance are in Figure 5.

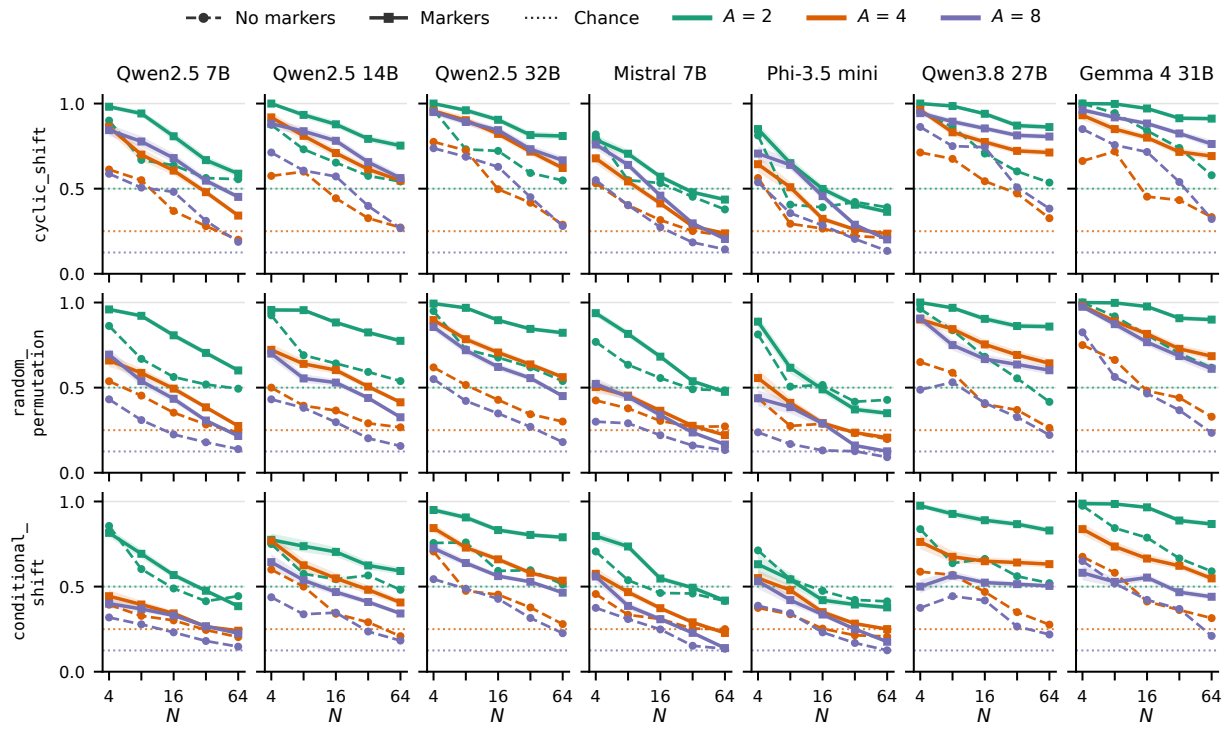


Figure 7: Position-local tasks on contiguous letters, by model and length. Character accuracy against sequence length N (4 to 64). Dashed: no markers; solid: markers (mean over spacings $k \in \{1, 2, 4, 8, 16\}$ with $k < N$); colors: alphabet size; dotted: chance; bands: ±1 standard error.

Mistral-7B. In all three position-local tasks, the benefit is largest at short lengths and decays to zero at the longest length tested (`cyclic_shift`, $N = 256$: +0.008, $z = 0.4$; `random_permutation`, $N = 64$:

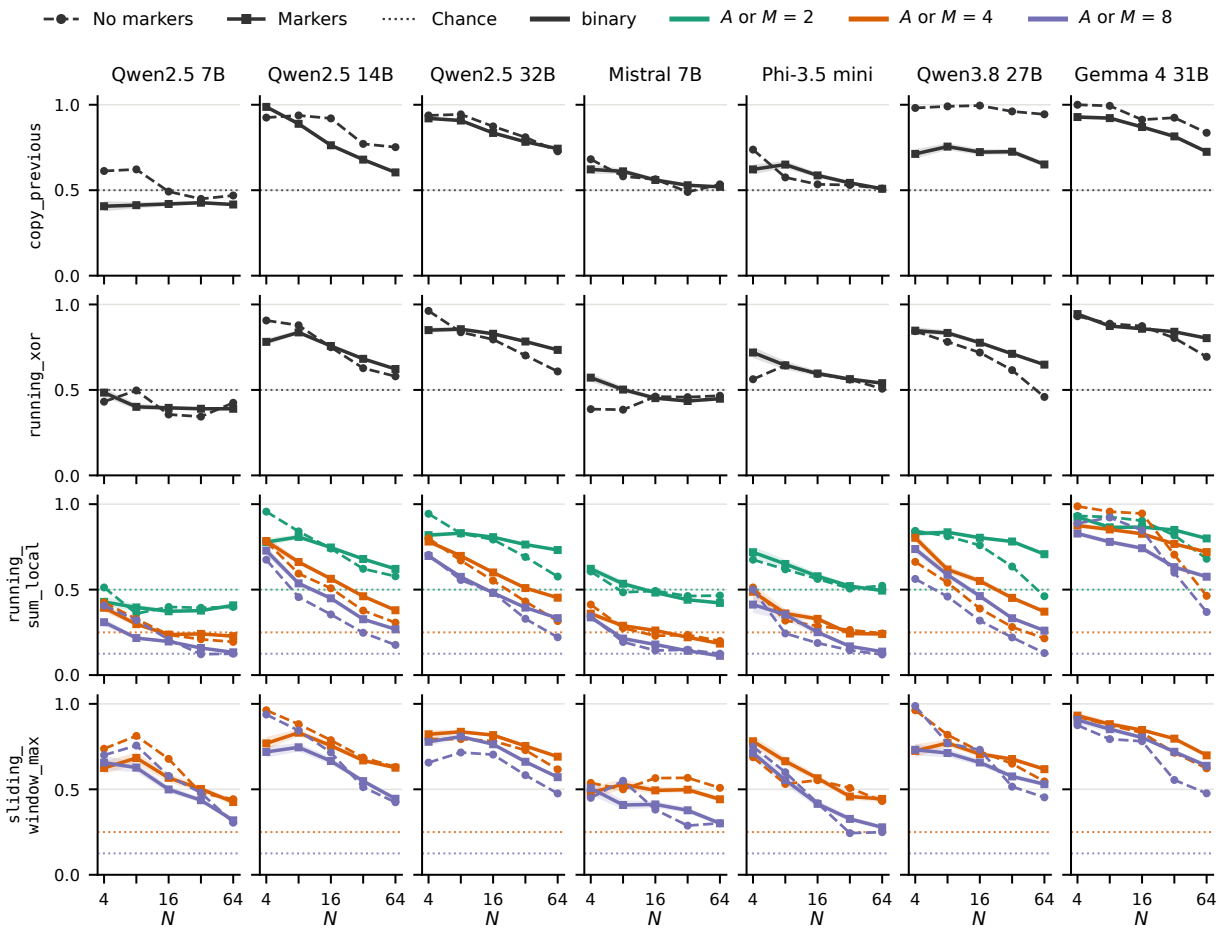


Figure 8: Dependency tasks (digits), by model and length; same conventions as Figure 7. Colors: modulus (`running_sum_local`) or alphabet size (`sliding_window_max`); black: binary. Marker runs average over all positions, so they mix the post-marker loss with the within-segment gain (Section 6).

-0.009, $z = 0.5$; `conditional_shift`, $N = 64$: -0.006, $z = 0.4$). In Qwen models the benefit holds or grows with length.

Repetition collapse. On `running_xor`, Qwen2.5-7B repeats a single wrong digit in about 38% of trials, more often under dense markers. This failure falls to 0.9% at 14B and 0.1% for Mistral-7B, so it is specific to Qwen2.5-7B.

Out-of-alphabet answers. On the digit tasks, Qwen2.5-7B often writes a letter (usually `t`, sometimes `o`) in place of `0`, in 34% to 39% of outputs, with or without markers. On spaced letters, several models step outside the alphabet (for example `I` after `H` instead of wrapping to `A`). Such characters are scored as errors.

E Per-position accuracy

Table 9 gives the per-position means plotted in Figure 3, and Table 10 breaks them down by distance from the marker. Table 11 repeats the dependency-task comparison under alignment-robust scoring, and Figure 9 shows the length profile of the tokenization controls for each task. "No markers" is the no-marker run's accuracy at the same position indices. Means are over the tasks in each block; per-task values are in `results_clean/_summary/per_position_values.csv`.

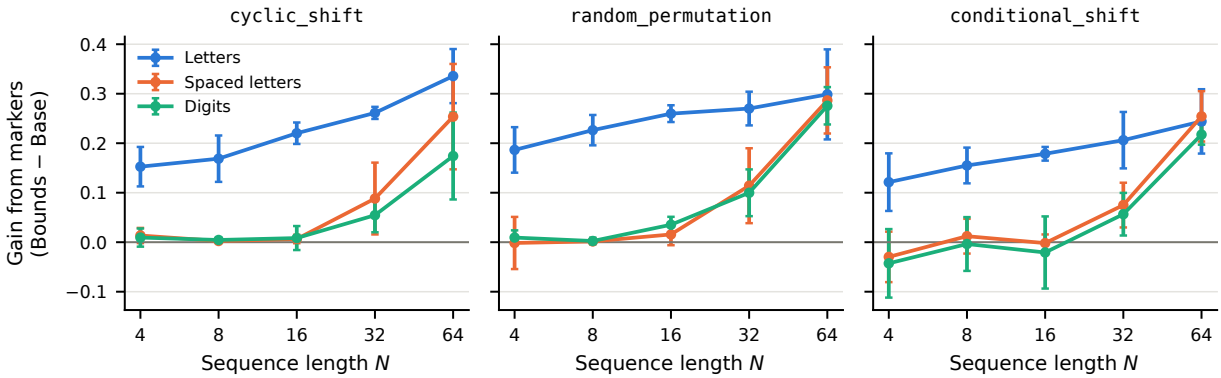


Figure 9: The length profile of Figure 4a, task by task (mean over Qwen2.5-14B and 32B, Qwen3.8-27B, and Gemma-4-31B; whiskers are 95% intervals over models). All three tasks show the same pattern.

Table 9: Per-position accuracy by model ($N \leq 64$, $k \in \{2, 4, 8, 16\}$), mean over the tasks in each block.

	Within a segment			Right after a marker		
Model	No markers	Markers	Δ	No markers	Markers	Δ
Position-local tasks, letters						
Qwen2.5 7B	0.38	0.45	+0.07	0.37	0.51	+0.14
Qwen2.5 14B	0.43	0.55	+0.11	0.40	0.63	+0.23
Qwen2.5 32B	0.49	0.65	+0.16	0.45	0.79	+0.34
Mistral 7B	0.35	0.35	0.00	0.30	0.39	+0.09
Phi-3.5 mini	0.31	0.31	0.00	0.28	0.35	+0.07
Qwen3.8 27B	0.50	0.69	+0.20	0.47	0.81	+0.34
Gemma 4 31B	0.56	0.72	+0.16	0.51	0.80	+0.29
Gemini 3.5 Flash	0.57	0.74	+0.17	0.52	0.83	+0.32
Position-local tasks, spaced letters						
Qwen2.5 7B	0.54	0.55	+0.01	0.52	0.52	0.00
Qwen2.5 14B	0.69	0.80	+0.11	0.67	0.82	+0.15
Qwen2.5 32B	0.79	0.93	+0.14	0.79	0.92	+0.14
Mistral 7B	0.40	0.43	+0.03	0.35	0.45	+0.09
Phi-3.5 mini	0.35	0.35	−0.01	0.33	0.34	+0.01
Qwen3.8 27B	0.83	0.92	+0.09	0.84	0.92	+0.08
Gemma 4 31B	0.88	0.94	+0.06	0.89	0.94	+0.05
Position-local tasks, digits						
Qwen2.5 7B	0.35	0.38	+0.04	0.30	0.35	+0.05
Qwen2.5 14B	0.77	0.85	+0.08	0.77	0.87	+0.09
Qwen2.5 32B	0.83	0.94	+0.11	0.83	0.93	+0.09
Mistral 7B	0.43	0.38	−0.04	0.41	0.39	−0.02
Phi-3.5 mini	0.48	0.55	+0.07	0.44	0.56	+0.12
Qwen3.8 27B	0.85	0.94	+0.09	0.85	0.94	+0.09
Gemma 4 31B	0.92	0.99	+0.07	0.92	0.98	+0.06
Dependency tasks						
Qwen2.5 7B	0.43	0.41	−0.02	0.42	0.36	−0.06
Qwen2.5 14B	0.68	0.68	0.00	0.64	0.49	−0.15
Qwen2.5 32B	0.70	0.77	+0.07	0.66	0.52	−0.15
Mistral 7B	0.44	0.43	0.00	0.43	0.38	−0.04
Phi-3.5 mini	0.49	0.48	0.00	0.45	0.42	−0.03
Qwen3.8 27B	0.68	0.70	+0.02	0.63	0.44	−0.19
Gemma 4 31B	0.79	0.85	+0.05	0.78	0.52	−0.26
Gemini 3.5 Flash	0.92	0.97	+0.05	0.92	0.49	−0.43
Dependency tasks, reset variants						
Qwen2.5 7B	0.43	0.43	0.00	0.41	0.58	+0.17
Qwen2.5 32B	0.69	0.79	+0.10	0.65	0.92	+0.27
Qwen3.8 27B	0.66	0.68	+0.03	0.63	0.98	+0.36
Gemma 4 31B	0.77	0.85	+0.08	0.75	0.97	+0.22

Table 10: Accuracy by distance after the marker at $k = 16$ ($N \in \{32, 64\}$): 0 is the position right after the marker. Mean over the tasks in each block.

		Positions after the marker				
Tasks	Model	0	1	2–3	4–7	8–15
Position-local, letters	Qwen2.5 7B	0.32	0.38	0.35	0.29	0.26
	Qwen2.5 14B	0.45	0.47	0.41	0.37	0.34
	Qwen2.5 32B	0.60	0.57	0.48	0.45	0.41
	Mistral 7B	0.28	0.34	0.29	0.26	0.25
	Phi-3.5 mini	0.30	0.28	0.28	0.28	0.23
	Qwen3.8 27B	0.68	0.67	0.59	0.51	0.47
	Gemma 4 31B	0.52	0.62	0.55	0.51	0.43
	Gemini 3.5 Flash	0.74	0.69	0.65	0.51	0.46
Position-local, spaced	Qwen2.5 7B	0.30	0.46	0.47	0.43	0.35
	Qwen2.5 14B	0.59	0.67	0.68	0.66	0.59
	Qwen2.5 32B	0.86	0.90	0.89	0.88	0.85
	Mistral 7B	0.31	0.39	0.37	0.27	0.24
	Phi-3.5 mini	0.25	0.37	0.33	0.25	0.21
	Qwen3.8 27B	0.89	0.88	0.91	0.91	0.90
	Gemma 4 31B	0.93	0.94	0.95	0.93	0.94
Position-local, digits	Qwen2.5 7B	0.28	0.38	0.32	0.30	0.25
	Qwen2.5 14B	0.83	0.85	0.86	0.81	0.76
	Qwen2.5 32B	0.90	0.93	0.94	0.93	0.90
	Mistral 7B	0.31	0.38	0.36	0.31	0.26
	Phi-3.5 mini	0.38	0.53	0.52	0.46	0.37
	Qwen3.8 27B	0.90	0.89	0.91	0.91	0.90
	Gemma 4 31B	0.94	0.98	0.99	0.98	0.98
Dependency	Qwen2.5 7B	0.36	0.41	0.42	0.42	0.36
	Qwen2.5 14B	0.45	0.75	0.68	0.60	0.53
	Qwen2.5 32B	0.49	0.85	0.79	0.71	0.63
	Mistral 7B	0.41	0.42	0.45	0.42	0.41
	Phi-3.5 mini	0.40	0.47	0.44	0.40	0.38
	Qwen3.8 27B	0.46	0.70	0.71	0.65	0.58
	Gemma 4 31B	0.49	0.82	0.83	0.81	0.75
	Gemini 3.5 Flash	0.57	1.00	1.00	1.00	0.97

Table 11: Per-position gains on dependency tasks (marker run minus the no-marker run at the same positions) under index-by-index scoring and under edit-distance alignment, which repairs dropped or inserted characters. $N \leq 64$, $k \in \{2, 4, 8, 16\}$.

		Right after a marker		Within a segment	
Task	Model	Index	Aligned	Index	Aligned
`copy_previous`	Qwen2.5 7B	−0.10	−0.09	−0.07	−0.06
	Qwen2.5 14B	−0.21	−0.15	−0.10	−0.05
	Qwen2.5 32B	−0.15	−0.17	−0.02	−0.02
	Mistral 7B	−0.05	−0.01	0.00	+0.01
	Phi-3.5 mini	−0.04	−0.07	−0.01	−0.04
	Qwen3.8 27B	−0.45	−0.20	−0.24	−0.05
	Gemma 4 31B	−0.29	−0.25	−0.07	−0.04
`running_xor`	Qwen2.5 7B	+0.02	−0.02	+0.01	0.00
	Qwen2.5 14B	−0.14	−0.13	+0.05	+0.01
	Qwen2.5 32B	−0.19	−0.24	+0.11	+0.03
	Mistral 7B	−0.04	−0.03	+0.02	−0.03
	Phi-3.5 mini	−0.05	−0.02	+0.03	+0.03
	Qwen3.8 27B	−0.06	−0.08	+0.13	+0.06
	Gemma 4 31B	−0.26	−0.29	+0.09	+0.04
`running_sum_local`	Qwen2.5 7B	−0.03	−0.04	−0.01	−0.02
	Qwen2.5 14B	−0.08	−0.12	+0.07	+0.03
	Qwen2.5 32B	−0.16	−0.22	+0.11	+0.05
	Mistral 7B	−0.01	−0.04	0.00	−0.01
	Phi-3.5 mini	−0.03	0.00	−0.02	−0.03
	Qwen3.8 27B	−0.06	−0.13	+0.14	+0.09
	Gemma 4 31B	−0.41	−0.46	+0.11	+0.07
`sliding_window_max`	Qwen2.5 7B	−0.11	−0.06	−0.02	−0.02
	Qwen2.5 14B	−0.16	−0.10	−0.03	−0.01
	Qwen2.5 32B	−0.09	−0.07	+0.08	+0.07
	Mistral 7B	−0.07	0.00	−0.02	0.00
	Phi-3.5 mini	+0.02	−0.03	−0.02	−0.03
	Qwen3.8 27B	−0.18	−0.16	+0.05	+0.04
	Gemma 4 31B	−0.07	−0.08	+0.08	+0.02

F Reset variants, pooled

Table 12 gives the pooled effect of markers on each dependency task in its original and reset forms; the per-position comparison is in Section 6.1.

Table 12: Reset variants, pooled over positions: Character accuracy Base → Bounds, Δ, and $|z|$, pooled as in Figure 5 (no-marker mean length-matched). Base differs between the original and reset columns because the reset prompt alone shifts no-marker accuracy; Section 6.1 therefore compares marker runs position by position. Originals use 20 to 80 trials per cell (Table 8); reset variants 20, except 80 at Qwen2.5-7B.

		Original (dependency intact)		Reset (dependency severed)	
Task	Model	Base→Bounds	Δ ($\|z\|$)	Base→Bounds	Δ ($\|z\|$)
`running_xor`	7B	0.401→0.403	+0.002 (0.1)	0.450→0.482	+0.032 (3.4)
	32B	0.746→0.798	+0.053 (6.2)	0.715→0.878	+0.163 (11.9)
	Qwen3.8 27B	0.646→0.742	+0.096 (9.5)	0.645→0.855	+0.210 (14.4)
	Gemma 4 31B	0.816→0.850	+0.034 (4.3)	0.816→0.928	+0.112 (10.2)
`running_sum_local`	7B	0.286→0.280	−0.006 (0.8)	0.277→0.348	+0.071 (12.4)
	32B	0.547→0.603	+0.056 (9.0)	0.508→0.660	+0.152 (12.1)
	Qwen3.8 27B	0.441→0.573	+0.131 (13.0)	0.419→0.683	+0.264 (19.4)
	Gemma 4 31B	0.754→0.775	+0.021 (3.2)	0.723→0.840	+0.117 (10.8)
`sliding_window_max`	32B	0.668→0.733	+0.065 (7.9)	0.677→0.816	+0.139 (10.9)
	Qwen3.8 27B	0.666→0.653	−0.013 (1.0)	0.640→0.824	+0.183 (16.4)
	Gemma 4 31B	0.708→0.783	+0.075 (6.3)	0.718→0.817	+0.099 (7.9)
`copy_previous`	7B	0.508→0.418	−0.090 (7.0)	0.565→0.553	−0.013 (1.4)
	32B	0.836→0.818	−0.018 (1.9)	0.874→0.792	−0.083 (4.6)
	Qwen3.8 27B	0.971→0.709	−0.262 (23.9)	0.942→0.836	−0.105 (6.4)
	Gemma 4 31B	0.917→0.832	−0.086 (6.5)	0.837→0.866	+0.029 (1.3)

G External tasks

Tables 13 and 14 and Figures 10 and 11 give the results on tasks we did not design (Section 6) and on Gemini.

Table 13: External position-local tasks, pooled across length and density. Base: no-marker accuracy averaged over lengths. Best-k: accuracy at the single best spacing (optimistic). Δ: marker runs minus the length-matched no-marker mean (Section 3). ProcBench-Substitute was regenerated after we made the vendored ProcBench generator's substitution table deterministic (it was drawn in a process-dependent order).

	Qwen2.5-7B			Qwen2.5-32B		
Task	Base	Best-k	Δ ($\|z\|$)	Base	Best-k	Δ ($\|z\|$)
Vowel/consonant tagging	0.438	0.435	−0.083 (2.4)	0.615	0.657	+0.012 (0.5)
ProcBench-Substitute	0.189	0.432	+0.096 (6.7)	0.390	0.757	+0.203 (10.8)
String copying	0.973	1.000	+0.009 (1.0)	0.993	1.000	+0.007 (2.0)

On Kadane's algorithm, base accuracy at both Qwen scales is statistically indistinguishable from chance under a shuffle test, so we do not interpret its effects. Minimum is well above chance at Qwen2.5-32B ($z = 4.1$ from shuffled chance).

Table 14: Closed-model check and CLRS-Text. Gemini 3.5 Flash with 3 trials per cell, pooled over all tested lengths (including N up to 256 for `cyclic_shift`), with the no-marker mean length-matched. CLRS-Text tasks use token-level markers and scoring.

Task	Model	Base acc.	Bounds acc.	Δ ($\lvert z \rvert$)
`cyclic_shift`	Gemini	0.596	0.852	+0.256 (14.4)
`random_permutation`	Gemini	0.561	0.846	+0.284 (8.4)
`conditional_shift`	Gemini	0.453	0.628	+0.175 (3.1)
`running_xor`	Gemini	0.882	0.913	+0.032 (0.4)
`running_sum_local`	Gemini	0.918	0.803	−0.116 (5.2)
`copy_previous`	Gemini	0.984	0.924	−0.060 (2.8)
CLRS minimum	Qwen2.5-7B	0.133	0.183	+0.050 (3.3)
	Qwen2.5-32B	0.592	0.392	−0.200 (4.5)
	Gemini	0.997	0.654	−0.343 (7.3)
CLRS Kadane	Qwen2.5-7B	0.060	0.068	+0.008 (1.0)
	Qwen2.5-32B	0.186	0.122	−0.064 (2.0)
	Gemini	0.384	0.354	−0.030 (0.3)

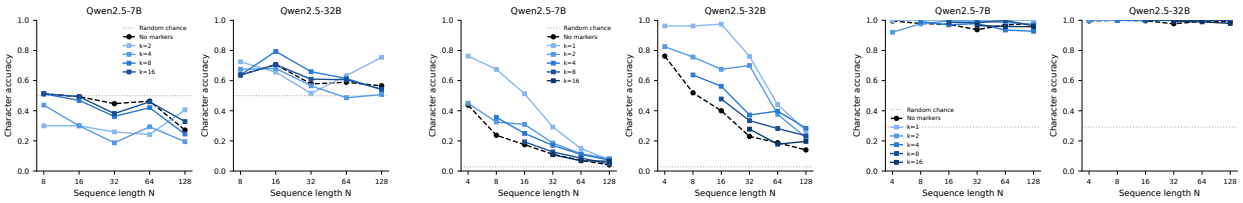


Figure 10: External tasks by marker density. Left: vowel/consonant tagging. Middle: ProcBench-Substitute. Right: string copying.

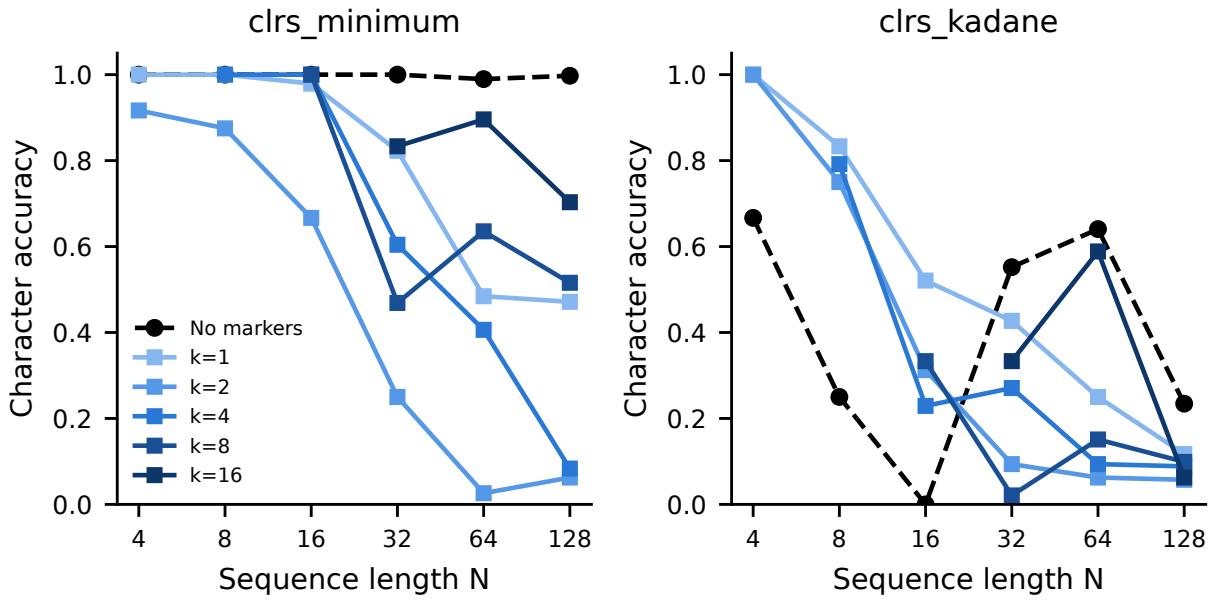


Figure 11: CLRS-Text tasks on Gemini 3.5 Flash by sequence length, one line per marker spacing k (3 trials per cell). Dashed: no markers.

H Mechanism details

This appendix collects the attention analyses behind Section 7: the mechanism overview (Figure 12), attention statistics by layer (Figure 13), example heads (Figure 14), the earlier head ablation (Figure 15), a digit-tokenization check of the attention pattern (Figure 16), error recovery after markers (Figure 17), the head taxonomy (Table 16), and the 7B ablation (Table 15). In Figure 17, the 50% rate away from markers is chance for this binary alphabet.

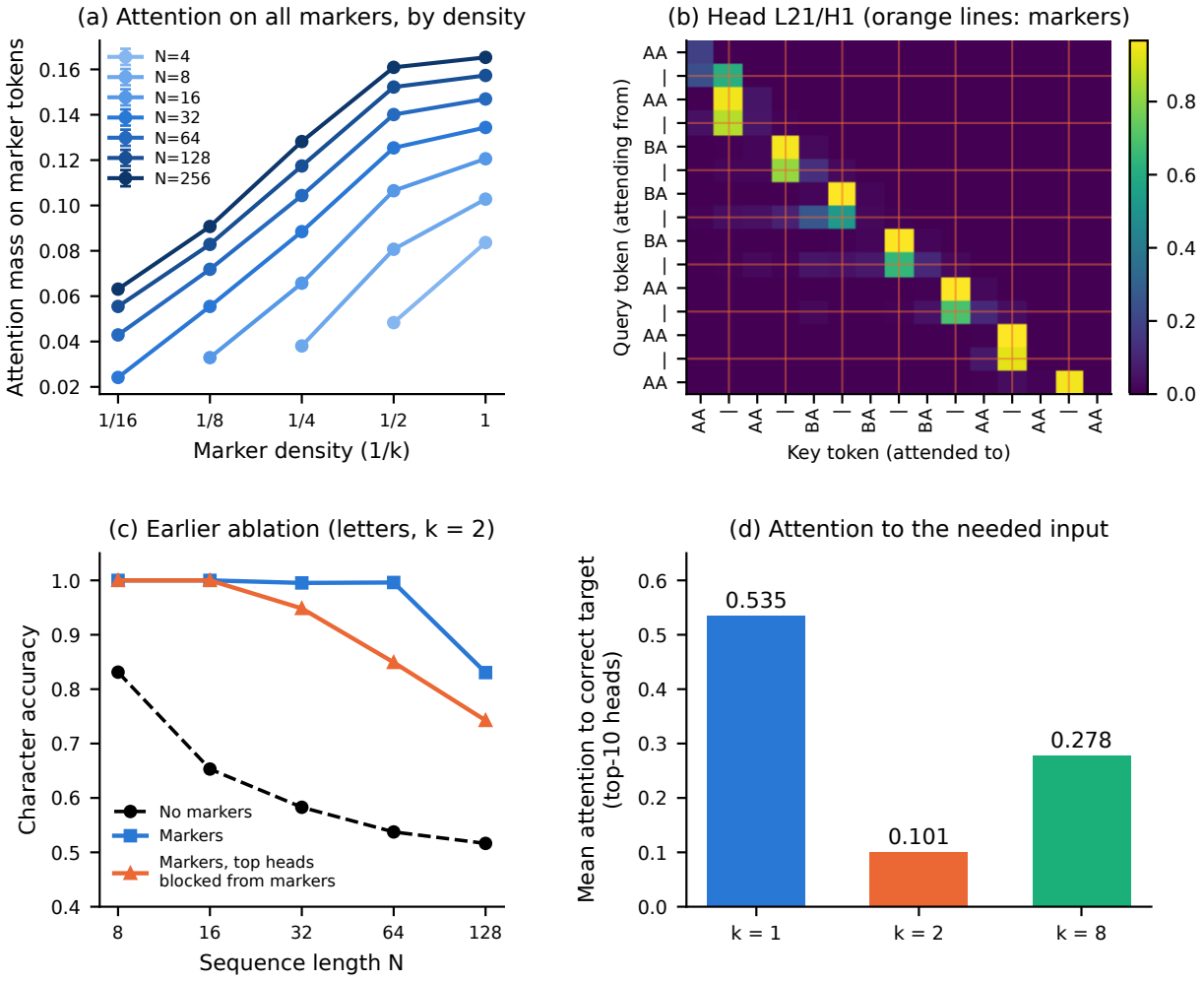


Figure 12: Mechanism (Qwen2.5-32B). (a) Total attention mass on all marker tokens rises with density at every length (`cyclic_shift`, $A = 2$); the number of markers rises faster, so this is not evidence that individual markers draw more attention than content tokens. (b) The most marker-attending head (layer 21, head 1) attends from each position to an adjacent marker. (c) Earlier ablation on contiguous letters ($k = 2$; Section 7): masking the ten most marker-attending heads' attention to markers removes a growing share of the benefit as N grows (0% at $N \leq 16$, 11% at 32, 32% at 64, 28% at 128). (d) `copy_previous` at $N = 32$: mean attention from each output position to the input token it needs (top-10 heads) is 0.535 at $k = 1$, 0.101 at $k = 2$, and 0.278 at $k = 8$, matching accuracy (Base 0.83; $k = 1$: 0.88; $k = 2$: 0.65; $k = 8$: 0.84).

Table 15: Head ablation at Qwen2.5-7B, same design as Table 3. Marker gains are small at this scale, so shares are noisy; "–" marks cells with a gain below 0.03.

				Markers, with heads blocked			Share of gain removed	
Format	N	No markers	Markers	Top 10	Random 10	Top 10, sham	Top 10	Controls
Letters	32	0.17	0.24	0.23	0.24	0.24	14%	0%
Letters	64	0.14	0.16	0.14	0.15	0.16	–	–
Spaced letters	32	0.42	0.55	0.51	0.52	0.55	26%	$\leq$ 22%
Spaced letters	64	0.19	0.30	0.27	0.31	0.32	24%	0%
Digits	32	0.24	0.27	0.28	0.29	0.28	–	–
Digits	64	0.18	0.19	0.19	0.19	0.19	–	–

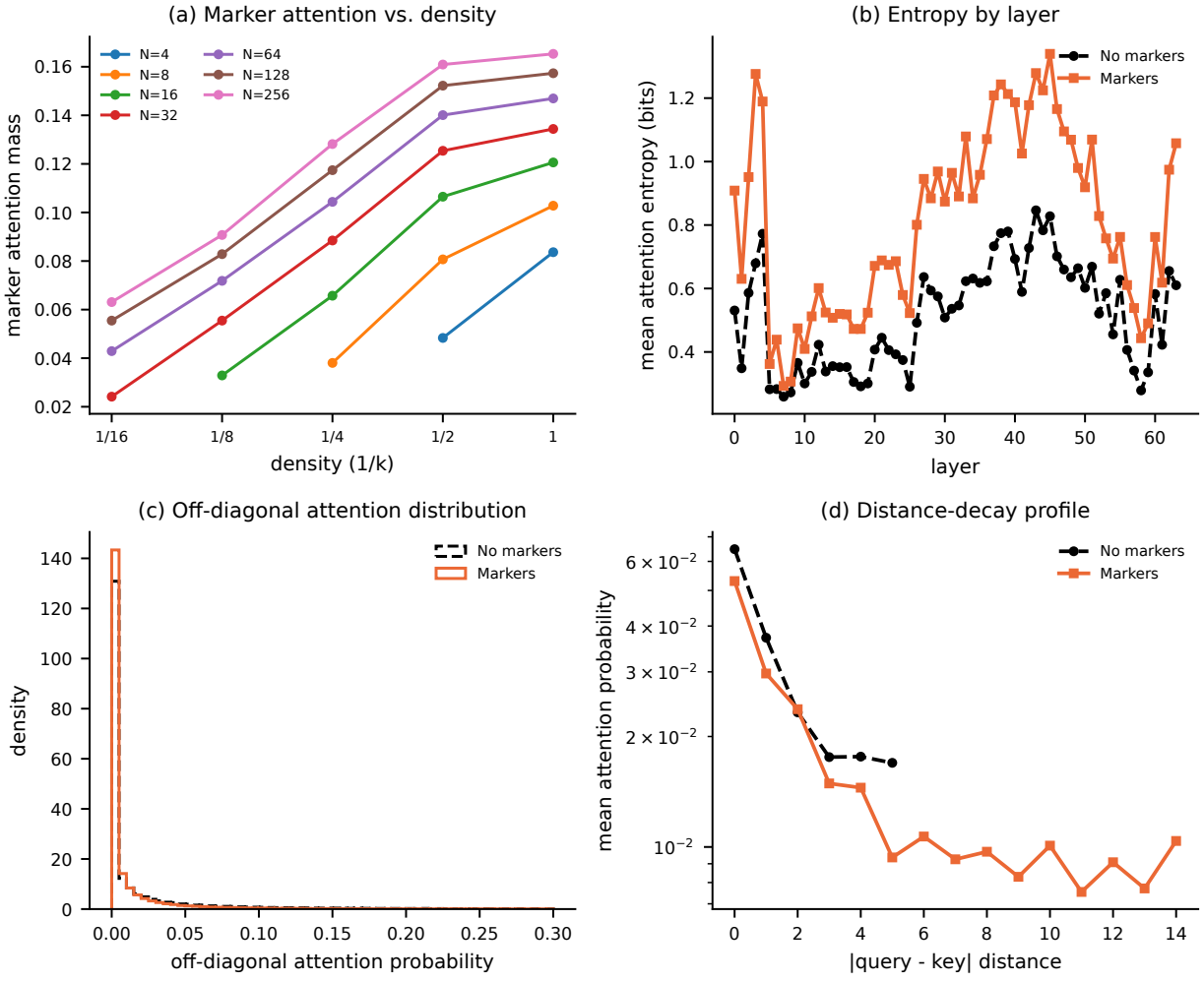


Figure 13: Attention analysis, Qwen2.5-32B, `cyclic_shift`: marker attention by density, attention entropy by layer (higher under markers at nearly every layer), off-diagonal attention, and distance decay.

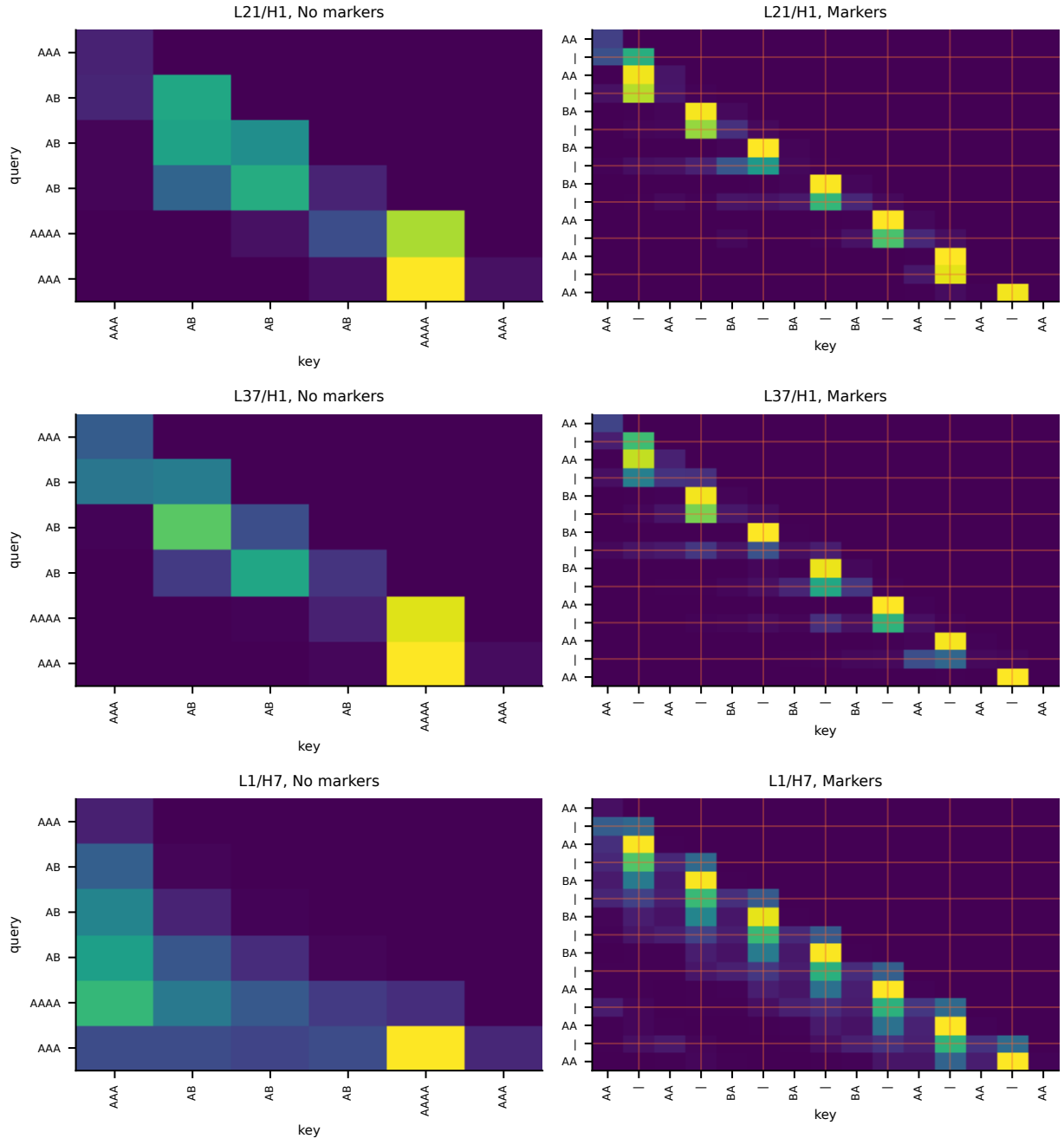


Figure 14: The three most marker-attending heads at Qwen2.5-32B (L21/H1, L37/H1, L1/H7). Without markers, letters merge into multi-character tokens; markers restore per-character resolution.

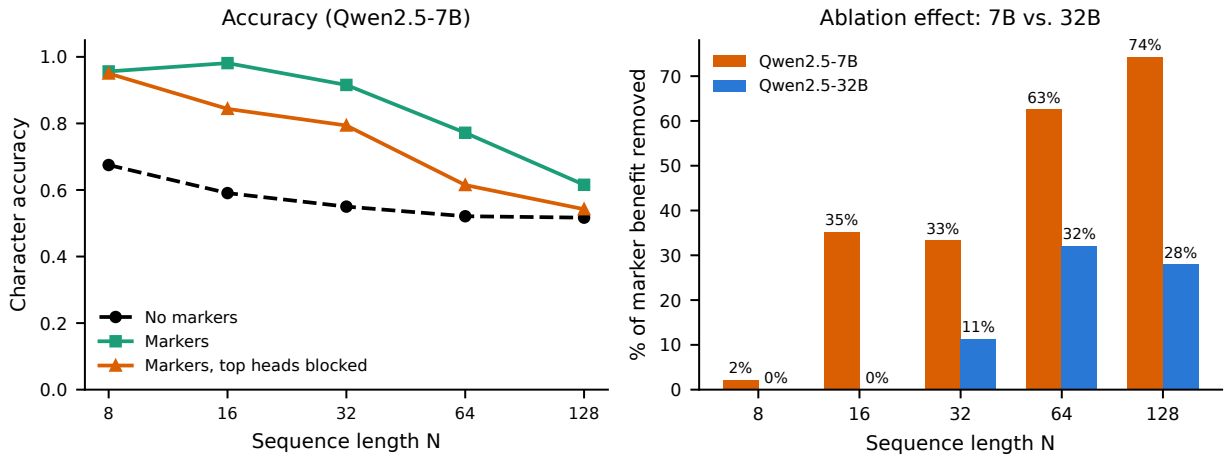


Figure 15: Earlier head ablation, contiguous letters (`cyclic_shift`, $A = 2$, $k = 2$), Qwen2.5-7B and 32B: share of the marker benefit removed by masking the ten most marker-attending heads' attention to markers, re-identified at each length. Unlike Table 3, Base and unablated runs come from an earlier grid and there are no controls at 32B.

Table 16: The ten most marker-attending heads at Qwen2.5-32B, with percentiles within all heads on an induction test and a positional (previous-token) test.

Layer/Head	Marker attention	Induction pct.	Positional pct.
21/1	0.099	38.3	98.3
37/1	0.087	47.4	96.7
1/7	0.086	17.1	97.7
35/19	0.082	23.1	96.5
35/3	0.080	14.7	97.2
30/32	0.079	18.6	96.0
32/37	0.075	14.3	98.8
1/8	0.072	64.8	97.7
32/7	0.071	60.4	90.2
19/19	0.068	9.9	83.9
Mean		30.9	95.3

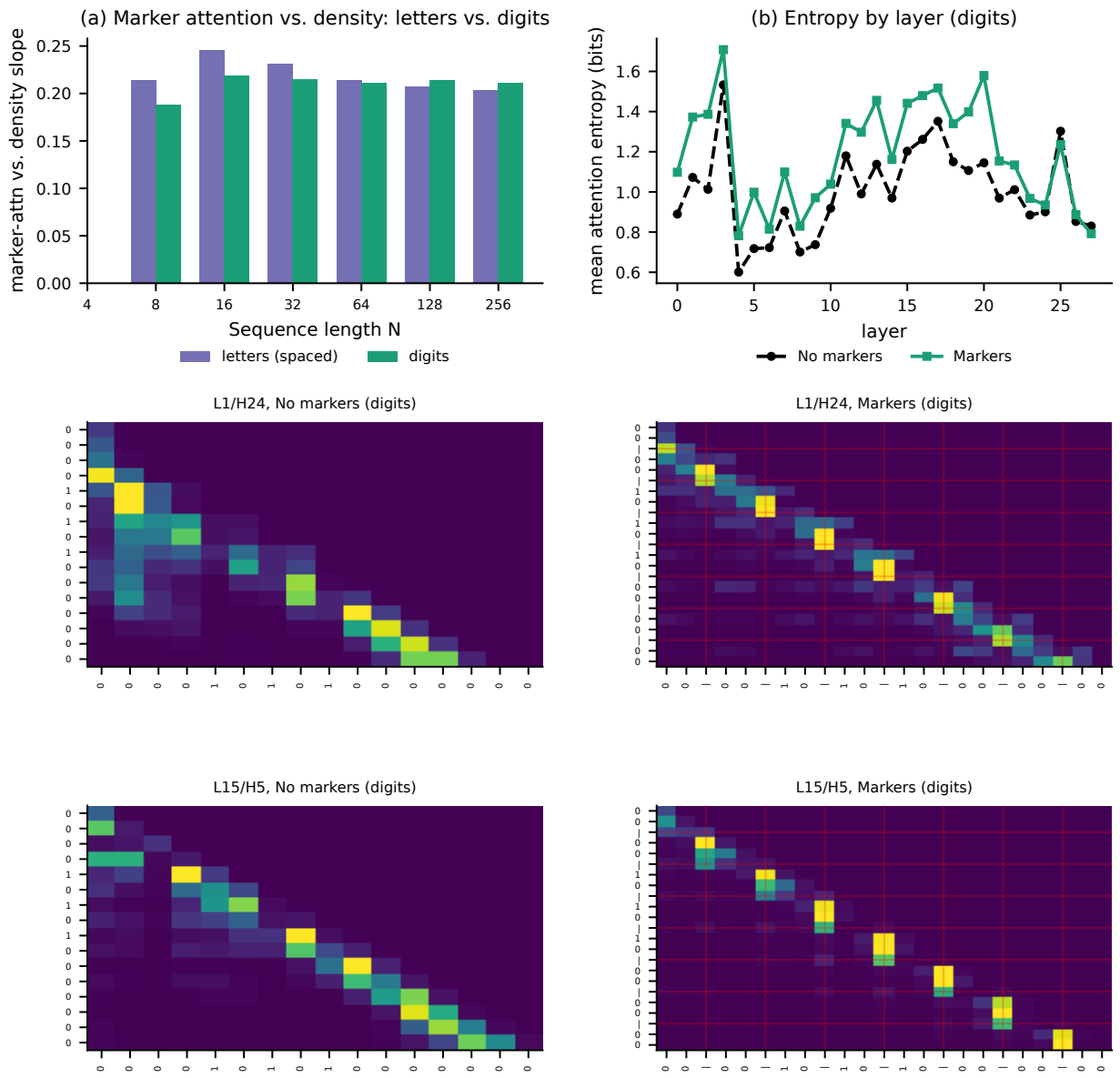


Figure 16: Tokenization check at Qwen2.5-7B: marker-attention slopes with letters (space-separated, one token per letter) and digits (natively one token per character) are nearly identical.

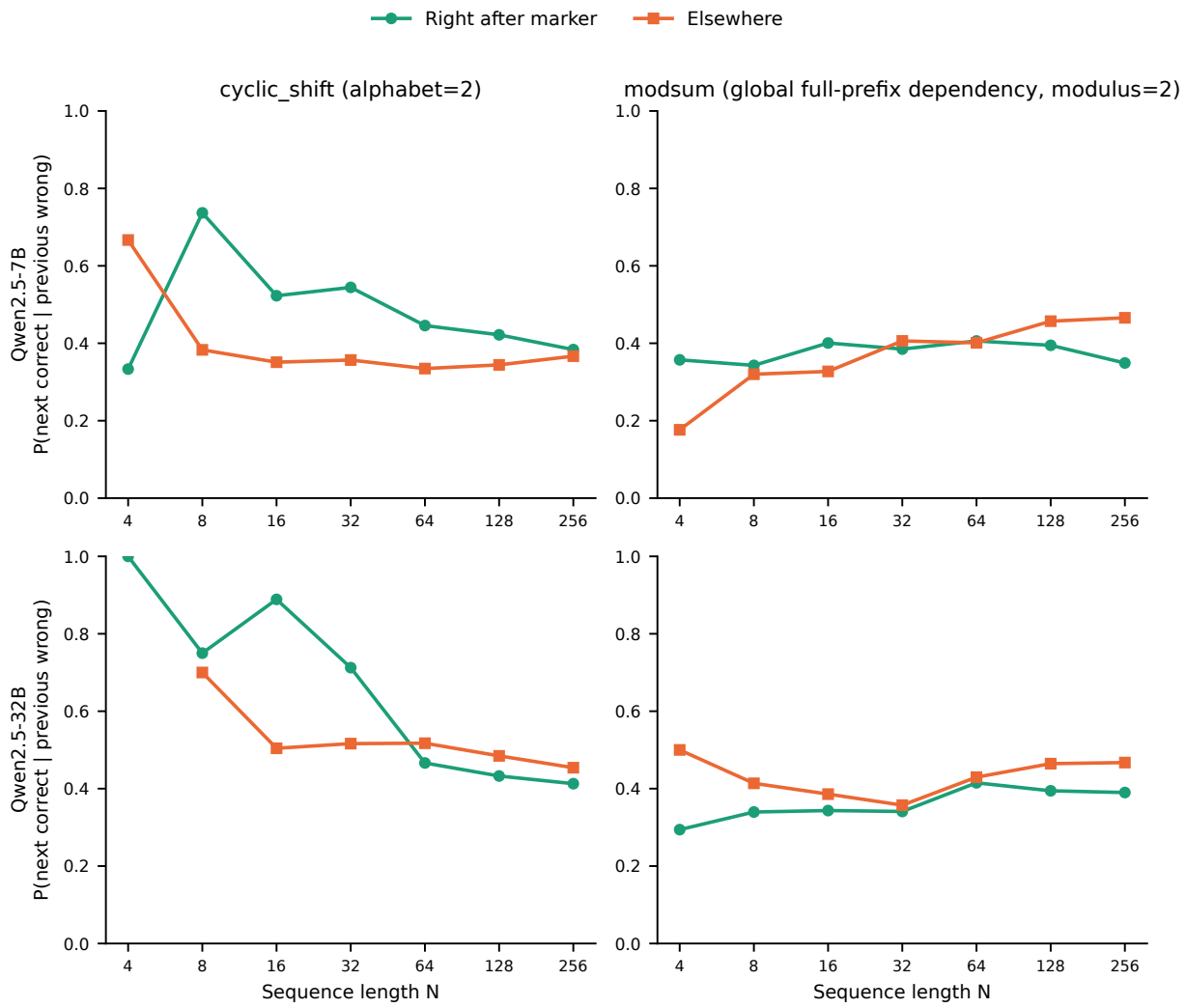


Figure 17: Error recovery: after a wrong output, the fraction of next positions that are correct, split by whether the next position follows a marker. Left: `cyclic_shift`; right: a full-prefix modular sum. $A = M = 2$.

I Marker ablations

Tables 17–19 vary the marker symbol, its placement, and the prompt wording on contiguous-letter `cyclic_shift` (Section 8).

Table 17: Marker identity, reserved token, and token budget (`cyclic_shift`, $A = 2$, periodic $k = 4$ unless noted). Character accuracy. Budget-matched: `<|>` at $k = 12$ uses the same total tokens as `|` at $k = 4$.

N	Model	Base	`|`	`;`	`#`	Reserved token	`<|>`, $k = 12$
8	7B	0.675	0.717	0.792	0.792	0.469	0.675
	32B	0.831	0.892	0.942	0.908	0.525	0.806
16	7B	0.591	0.825	0.750	0.667	0.216	0.591
	32B	0.653	0.896	0.846	0.912	0.284	0.759
32	7B	0.550	0.679	0.769	0.585	0.092	0.539
	32B	0.583	0.848	0.781	0.744	0.186	0.688
64	7B	0.521	0.616	0.650	0.591	0.049	0.453
	32B	0.537	0.759	0.658	0.693	0.160	0.710
128	7B	0.517	0.545	0.507	0.496	0.046	0.452
	32B	0.516	0.694	0.744	0.561	0.219	0.642

Table 18: Placement regularity (`cyclic_shift`, $A = 2$, marker count matched to $k = 4$). Character accuracy.

	Qwen2.5-7B					Qwen2.5-32B				
N	Base	Periodic	± 1	± 2	Random	Base	Periodic	± 1	± 2	Random
8	0.675	0.717	0.581	0.675	0.633	0.831	0.892	0.869	0.912	0.858
16	0.591	0.825	0.572	0.566	0.583	0.653	0.896	0.863	0.787	0.733
32	0.550	0.679	0.572	0.509	0.552	0.583	0.848	0.847	0.745	0.779
64	0.521	0.616	0.502	0.491	0.491	0.537	0.759	0.731	0.732	0.682
128	0.517	0.545	0.493	0.471	0.452	0.516	0.694	0.691	0.695	0.583

Table 19: Prompt wording (`cyclic_shift`, $A = 2$, $k = 4$, 15 trials per cell). Δ = Bounds − Base character accuracy.

	Qwen2.5-7B, $N =$					Qwen2.5-32B, $N =$				
Wording	8	16	32	64	128	8	16	32	64	128
Formal	+.09	+.27	+.10	+.13	+.07	+.26	+.17	+.17	+.20	+.18
Casual	−.04	+.08	+.12	+.04	+.10	+.08	+.25	+.17	+.21	+.20
Terse cipher	−.07	+.08	+.00	+.00	+.05	−.12	−.11	+.01	−.07	−.05

J Delimiters, longer sequences, sampling, and larger state spaces

Table 20 gives the follow-up experiment with commas and line breaks as markers and sequences up to $N = 256$, and Table 21 repeats three tasks with sampling instead of greedy decoding (Sections 5 and 6).

Table 20: Delimiter type and longer sequences at Qwen2.5-32B ($k \in \{4, 16\}$ pooled, 20 trials per cell; `random_permutation` with digits at $A = 8$, `running_sum_local` at modulus 4). Change in accuracy: marker run minus no-marker run, pooled over positions. Post-marker and within-segment: marker run minus the no-marker run at the same positions. Each delimiter is named in its own constraint sentence. The `|` cells at $N = 64$ reproduce the main grid exactly (same prompts, greedy decoding). No output reached the generation budget.

		Change in accuracy			Post-marker			Within-segment		
Task	N	`|`	`,`	line break	`|`	`,`	line break	`|`	`,`	line break
`random_permutation (digits)`	64	+0.52	+0.52	+0.53	+0.56	+0.56	+0.53	+0.52	+0.52	+0.53
	128	+0.59	+0.57	+0.59	+0.67	+0.64	+0.65	+0.58	+0.56	+0.58
	256	+0.46	+0.46	+0.47	+0.53	+0.53	+0.50	+0.46	+0.45	+0.46
`running_xor`	64	+0.15	+0.16	+0.17	−0.13	−0.11	−0.14	+0.20	+0.21	+0.24
	128	+0.18	+0.13	+0.21	−0.03	−0.05	−0.04	+0.22	+0.16	+0.26
	256	+0.17	+0.15	+0.20	+0.04	+0.03	+0.05	+0.19	+0.17	+0.23
`running_sum_local`	64	+0.13	+0.11	+0.15	−0.13	−0.11	−0.12	+0.17	+0.15	+0.19
	128	+0.11	+0.09	+0.14	+0.02	+0.00	+0.02	+0.14	+0.11	+0.17
	256	+0.08	+0.07	+0.12	+0.03	+0.02	+0.05	+0.09	+0.09	+0.14

Table 21: Sampling instead of greedy decoding at Qwen2.5-32B ($N \in \{32, 64\}$, $k \in \{4, 16\}$ pooled, 20 prompts per cell, one sample each; `random_permutation` at $A = 8$, `running_sum_local` at modulus 4). The greedy rows are the main-grid runs on the same prompts. Change: marker run minus the no-marker run of the same length and trial, with a 95% interval. Post-marker and within-segment: marker run minus the no-marker run of the same decoding regime at the same positions.

Task	Decoding	No markers	Markers	Change	Post-marker	Within-segment
`random_permutation (digits)`	Greedy	0.55	0.92	$+0.37 \pm 0.05$	—	—
	$T = 0.7$, top-p 0.8	0.54	0.92	$+0.38 \pm 0.05$	—	—
	$T = 1.0$, top-p 0.95	0.52	0.93	$+0.42 \pm 0.05$	—	—
`running_xor`	Greedy	0.66	0.77	$+0.11 \pm 0.03$	−0.18	+0.16
	$T = 0.7$, top-p 0.8	0.67	0.76	$+0.09 \pm 0.03$	−0.16	+0.14
	$T = 1.0$, top-p 0.95	0.66	0.76	$+0.10 \pm 0.03$	−0.17	+0.15
`running_sum_local`	Greedy	0.39	0.48	$+0.09 \pm 0.03$	−0.04	+0.12
	$T = 0.7$, top-p 0.8	0.39	0.47	$+0.08 \pm 0.03$	−0.06	+0.12
	$T = 1.0$, top-p 0.95	0.39	0.47	$+0.08 \pm 0.03$	−0.07	+0.11

Table 22: Larger state spaces at Qwen2.5-32B: `running_sum_local` at modulus M, with the symbols written space-separated so each is one token at every M ($N \in \{32, 64\}$, $k \in \{4, 16\}$ pooled, 20 trials per cell). Chance is $1/M$. Change: marker run minus no-marker run, with a 95% interval over generations. Post-marker gives the accuracy at that position with its change against the no-marker run at the same positions in parentheses; within-segment gives that change for the other positions. The $M = 8$ rows are not comparable with the main grid's $M = 8$ cells, which use unspaced digits.

M	Chance	No markers	Markers	Change	Post-marker	Within-segment
8	0.125	0.22	0.26	$+0.04 \pm 0.03$	0.13 (−0.07)	+0.06
16	0.062	0.14	0.20	$+0.06 \pm 0.03$	0.05 (−0.06)	+0.08
32	0.031	0.07	0.10	$+0.02 \pm 0.02$	0.04 (−0.02)	+0.03

Table 23: Prompt delivery at Qwen2.5-32B ($N \in \{32, 64\}$, $k \in \{4, 16\}$ pooled, 20 trials per cell). Default: the chat template with no system message supplied, which makes the template insert its own; the main grid's runs, on the same prompts and seeds. Empty: a system message of `""`, which suppresses that default. Task-oriented: "You are a precise, deterministic text transformer…". No chat template: the raw prompt as a completion. Change has a 95% interval over generations; post-marker and within-segment are against the no-marker run of the same condition at the same positions.

Task	Prompt delivery	No markers	Markers	Change	Post-marker	Within-s
`running_xor`	Default template	0.66	0.77	$+0.11 \pm 0.04$	-0.18	$+0.$
	Empty system message	0.66	0.78	$+0.12 \pm 0.04$	-0.17	$+0.$
	Task-oriented system message	0.68	0.77	$+0.09 \pm 0.04$	-0.14	$+0.$
	No chat template	0.66	0.80	$+0.13 \pm 0.04$	-0.15	$+0.$
`random_permutation (digits)`	Default template	0.55	0.92	$+0.37 \pm 0.07$	—	—
	Empty system message	0.59	0.95	$+0.36 \pm 0.06$	—	—
	Task-oriented system message	0.55	0.94	$+0.39 \pm 0.06$	—	—
	No chat template	0.60	0.95	$+0.36 \pm 0.07$	—	—

K Accuracy by marker density

Figure 18 shows accuracy as a function of marker density at each length, and Table 24 tests whether a simple curve describes it.

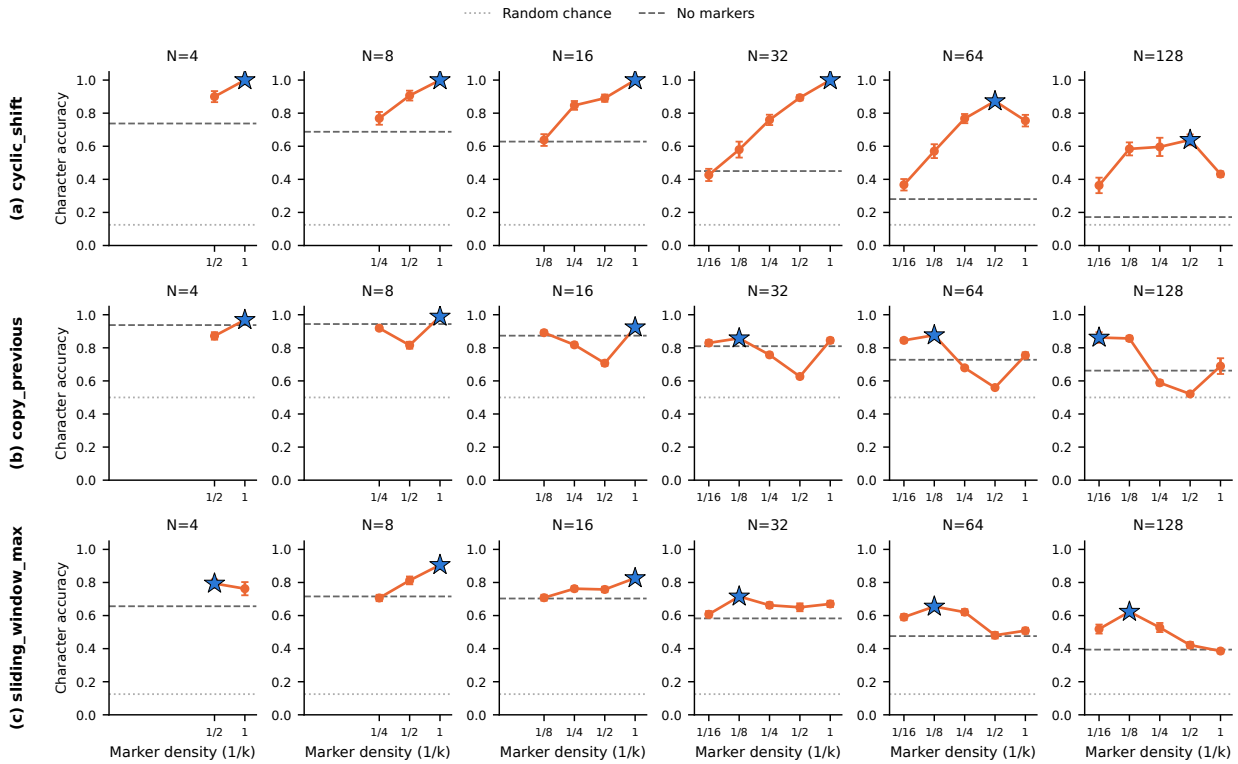


Figure 18: Accuracy by marker density at each length, contiguous letters and dependency tasks (Qwen2.5-32B; $A = 8$ where applicable). Dashed: no-marker baseline; dotted: chance; star: best density.

Table 24: Does a quadratic in density describe accuracy curves? "AIC": share of curves where a quadratic beats linear and exponential fits in-sample. "Held-out": share where it also beats a linear fit on leave-one-density-out prediction.

Task	Tier	Curves	Mean quadratic R^2	AIC	Held-out
`cyclic_shift`	position-local	15	0.876	93%	47%
`random_permutation`	position-local	12	0.941	92%	42%
`conditional_shift`	position-local	12	0.908	100%	50%
`copy_previous`	one-step	4	0.608	50%	0%
`running_xor`	one-step	4	0.449	25%	0%
`running_sum_local`	one-step	12	0.512	42%	0%
`sliding_window_max`	windowed	8	0.473	50%	0%

L Well-formedness and cost

Tables 26 and 27 and Figure 19 give output well-formedness and the time cost of markers (Section 8).

Character accuracy strips markers and spaces before comparing, so it does not say whether the model followed the whole instruction. Table 25 adds two stricter measures over the 86,400 generations whose raw text we stored: *exact transformation match*, the share whose output equals the answer once markers and spaces are removed from both sides, and *compliant*, the share whose raw first line carries markers in exactly the right places (for no-marker runs, carries none). Markers raise exact match on the position-local tasks, from 0.16 to 0.38 on contiguous letters, 0.39 to 0.43 on digits, and 0.39 to 0.42 on spaced letters, and lower it on the dependency tasks, from 0.24 to 0.11, the same directions as character accuracy. Marker placement is followed in 95% of marked generations at Gemma 4, 94% at Qwen3.8, and 92% at Qwen2.5-32B, against 60% to 62% at Mistral, Phi, and Qwen2.5-7B; without markers, models almost never emit one (94% to 100% compliant). So at the stronger models, where the paper's main claims are made, format failures are too rare to account for the accuracy differences, while at the three smallest models a substantial minority of marked outputs do not carry the markers as asked.

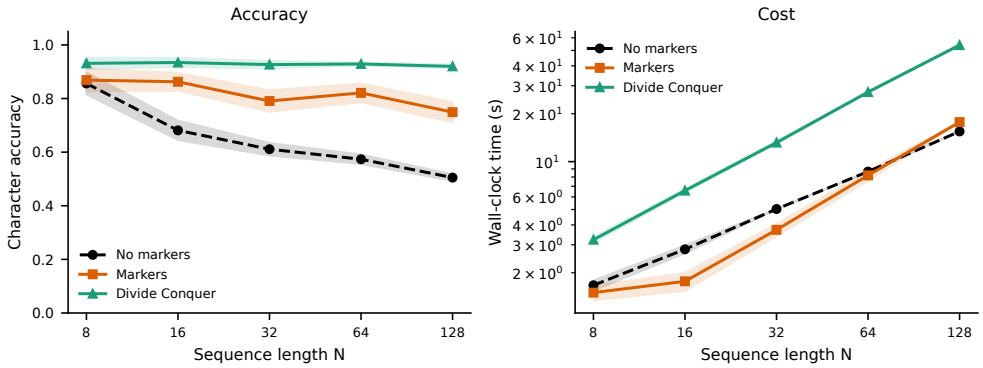


Figure 19: Markers versus true divide-and-conquer (one call per segment), Qwen2.5-32B: accuracy and wall-clock by length.

Table 25: Exact transformation match (markers and spaces removed from both sides) and marker-placement compliance by model and task group ($N \leq 64$, all spacings pooled). Exact: the cleaned output equals the answer. Compliant: the raw first line carries markers at exactly the expected offsets, or none for the no-marker runs.

		No markers		Markers	
Model	Task group	Exact	Compliant	Exact	Compliant
Qwen2.5 7B	Position-local, spaced letters	0.27	0.98	0.17	0.68
	Position-local, digits	0.03	0.96	0.03	0.65
	Dependency, as is	0.12	1.00	0.03	0.26
Qwen2.5 14B	Position-local, spaced letters	0.42	1.00	0.40	0.82
	Position-local, digits	0.47	1.00	0.47	0.94
	Dependency, as is	0.21	1.00	0.07	0.65
Qwen2.5 32B	Position-local, spaced letters	0.54	1.00	0.70	0.93
	Position-local, digits	0.59	1.00	0.71	0.95
	Dependency, as is	0.04	1.00	0.09	0.71
Mistral 7B	Position-local, spaced letters	0.11	0.98	0.10	0.55
	Position-local, digits	0.10	0.94	0.07	0.75
	Dependency, as is	0.03	0.95	0.01	0.28
Phi-3.5 mini	Position-local, spaced letters	0.07	0.94	0.05	0.50
	Position-local, digits	0.12	0.91	0.11	0.76
	Dependency, as is	0.06	0.44	0.05	0.48
Qwen3.8 27B	Position-local, letters	0.13	1.00	0.35	0.93
	Position-local, spaced letters	0.57	1.00	0.71	0.97
	Position-local, digits	0.61	1.00	0.75	0.98
	Dependency, as is	0.24	1.00	0.10	0.89
Gemma 4 31B	Position-local, letters	0.20	1.00	0.41	0.89
	Position-local, spaced letters	0.72	1.00	0.81	0.99
	Position-local, digits	0.78	1.00	0.88	0.99
	Dependency, as is	0.38	1.00	0.17	0.94

Table 26: Well-formedness at Qwen2.5-32B. VVR: share of outputs using only the task alphabet. CR: share at least as long as the answer. WFR: share satisfying both. Pooled across lengths and alphabets; first seed batch only, so trial counts are half of Table 8 for the doubly-run tasks.

Task	Condition	Trials	Acc.	VVR	CR	WFR
`cyclic_shift`	Base	420	0.511	0.536	0.919	0.521
	Bounds	1740	0.701	0.568	0.806	0.444
`random_permutation`	Base	600	0.499	0.995	0.858	0.855
	Bounds	2280	0.720	0.994	0.923	0.918
`conditional_shift`	Base	300	0.514	0.917	0.937	0.863
	Bounds	1140	0.671	0.834	0.908	0.752
`running_xor`	Base	200	0.781	1.000	0.940	0.940
	Bounds	760	0.796	1.000	0.930	0.930
`running_sum_local`	Base	600	0.590	0.830	0.900	0.765
	Bounds	2280	0.601	0.699	0.946	0.675
`copy_previous`	Base	200	0.859	1.000	0.955	0.955
	Bounds	760	0.817	1.000	0.879	0.879
`sliding_window_max`	Base	200	0.684	1.000	0.995	0.995
	Bounds	760	0.727	1.000	0.943	0.943

Table 27: Wall-clock seconds per call at Qwen2.5-32B. Throughput is 14 to 15 tokens/s in every condition.

Condition	k	$N = 8$	16	32	64	128
Base	–	0.330	0.570	1.270	3.763	11.380
Bounds	1	1.133	2.215	4.403	7.684	14.260
Bounds	2	0.544	1.076	2.142	4.213	9.000
Bounds	4	0.380	0.787	1.729	3.298	6.453
Bounds	8	0.312	0.613	1.288	2.479	4.976